DSA Complete Guide

50+ Algorithmic Problems

Deep Explanations · Golang Code · Real-World Context

Arrays	Strings	Linked Lists	Stack	Queue

- **Pattern Recognition**
  Every problem mapped to its core algorithmic pattern
- **Step-by-Step Dry Runs**
  Trace through examples index by index
- **Real-World Connections**
  How each algorithm powers production systems
- **Time & Space Analysis**
  Complexity breakdowns for every solution
- **Golang Code**
  Clean, idiomatic Go implementations throughout
- **Common Mistakes**
  Pitfalls and edge cases explicitly highlighted

NeetCode 75 · LeetCode Patterns · Backend Engineering Focus

Table of Contents

Arrays

- P1: Contains Duplicate
- P2: Valid Anagram
- P3: Two Sum
- P4: Group Anagrams
- P5: Top K Frequent Elements
- P6: Product of Array Except Self
- P7: Valid Sudoku
- P8: Encode and Decode Strings
- P9: Longest Consecutive Sequence
- P10: Best Time to Buy & Sell Stock
- P11: Container With Most Water
- P12: 3Sum
- P13: Trapping Rain Water

Strings

- P14: Valid Palindrome
- P15: Longest Substring Without Repeating Characters
- P16: Longest Repeating Character Replacement
- P17: Longest Palindromic Substring
- P18: Palindromic Substrings Count
- P19: Minimum Window Substring
- P20: Word Break
- P21: Word Search

Linked Lists

- P22: Reverse Linked List
- P23: Merge Two Sorted Lists
- P24: Linked List Cycle
- P25: Remove Nth From End
- P26: Reorder List
- P27: Add Two Numbers
- P28: Linked List Cycle II
- P29: Find Duplicate Number
- P30: Copy List With Random Pointer
- P31: LRU Cache

Stack

- P32: Valid Parentheses
- P33: Min Stack

- P34: Evaluate Reverse Polish Notation
- P35: Generate Parentheses
- P36: Daily Temperatures
- P37: Car Fleet
- P38: Largest Rectangle in Histogram

Queue

- P39: Implement Queue Using Stacks
- P40: Sliding Window Maximum
- P41: Binary Tree Level Order Traversal
- P42: Binary Tree Right Side View
- P43: Rotting Oranges

How to Use This Guide

Each problem in this guide is structured to build genuine understanding — not just memorization. Here's how to approach each section for maximum learning:

1. Real-World Analogy First

Before looking at code, read the analogy. Ask yourself: where have I seen this in production systems? This activates existing mental models and makes patterns sticky.

2. Constraints Analysis

Read constraints BEFORE writing any code. Constraints tell you the allowed time complexity, hint at data structures (sorted → binary search, small k → heap), and reveal edge cases.

3. Brute Force → Optimization

Always understand the naive $O(n^2)$ or $O(n^3)$ solution first. The optimization is a direct response to its bottleneck. You'll remember optimizations 10x better when you know what problem they fix.

4. Trace the Dry Run

Cover the code and manually trace the dry run step by step. If you can reproduce the trace, you understand the algorithm. If not, reread the intuition section.

5. Pattern → Multiple Problems

Each problem belongs to a pattern (sliding window, two pointers, monotonic stack, etc.). Once you recognize the pattern, 10+ other problems become approachable immediately.

6. Code It Without Looking

After understanding, close the guide and implement in your language of choice. Compare with the Golang solution — the logic should be identical even if syntax differs.

> ■ **Insight**
> Study tip: Focus on ONE pattern per day. Do all problems in a category before moving on. Spaced repetition beats marathon sessions — revisit problems 3 days, 1 week, and 1 month later.

■ ARRAYS

Problems 1–13 · Hash Maps · Two Pointers · Sliding Window

1 Contains Duplicate

■ 1. Problem Understanding

Given an integer array **nums**, return **true** if any value appears at least twice in the array, and return **false** if every element is distinct.

Real-World Analogy

Imagine you're a security guard at a concert checking wristbands. Each wristband has a unique serial number. As people enter, you must detect if anyone tries to sneak in twice using the same wristband. You scan each wristband — if you've seen this number before, raise the alarm. This is exactly what 'Contains Duplicate' does: scan a list, detect if any item was seen before.

What Problem Does This Solve?

- Database integrity: detecting duplicate primary keys before INSERT
- User registration: checking if an email/username already exists
- File deduplication: detecting duplicate files before backup
- Log analysis: finding repeated error codes in a stream

What If This Problem Didn't Exist?

Without duplicate detection, databases would insert duplicate rows silently, causing data corruption. Systems would charge users twice, register same emails, overwrite critical records. Every modern database engine has uniqueness checks built at the core — this problem is that fundamental.

Input / Output

- Input: nums = [1, 2, 3, 1] → Output: true (1 appears twice)
- Input: nums = [1, 2, 3, 4] → Output: false (all unique)
- Input: nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2] → Output: true

■ 2. Constraints Analysis

- 1 ≤ nums.length ≤ 10^5 → Up to 100,000 elements; $O(n^2)$ is too slow
- -10^9 ≤ nums[i] ≤ 10^9 → Wide range; cannot use array indexing as hash
- Edge case: single element array → always false (no duplicates possible)
- Edge case: all same elements → true immediately on second element

- Edge case: negatives → sorting still works; hashmap handles negatives fine

■ 3. Brute Force Approach

Intuition: A beginner sees this and thinks: 'For every element, check every OTHER element — if any two are equal, return true.' This is the most natural thought: compare all pairs.

Step-by-step Breakdown

- Use two nested loops: outer loop index i, inner loop index j
- For each i, check all j > i (to avoid comparing element with itself)
- If nums[i] == nums[j], we found a duplicate → return true
- If no pair found after all comparisons → return false

Dry Run

nums = [1, 2, 3, 1]

i=0 (val=1): j=1→2≠1, j=2→3≠1, j=3→1==1 ✓ → return true

Golang — Brute Force

```
// Brute Force — O(n²) time

func containsDuplicate(nums []int) bool {

n := len(nums)

for i := 0; i &lt; n; i++ {

for j := i + 1; j &lt; n; j++ {

if nums[i] == nums[j] {

return true

}

}

}

return false

}
```

■ Time Complexity	■ Space Complexity
O(n²)	**O(1)**

For each of n elements, we scan up to n-1 others. With n=100,000, that's 5 billion comparisons — painfully slow.

Why Inefficient: We re-examine elements we've already seen. There's no memory of what we've processed. We need a way to 'remember' past elements in O(1) lookup time.

■ 4. Optimized Approach — HashMap

Key Insight: Instead of comparing every pair, maintain a set of elements seen so far. For each new element, check if it's already in the set — O(1) average lookup. If yes, duplicate found. If no, add to set and continue.

Step-by-step Explanation

- Create an empty hash set 'seen'
- Iterate through each number in nums
- If the number is already in 'seen' → return true (duplicate!)
- Otherwise, add the number to 'seen' and continue
- If loop completes without finding duplicate → return false

Dry Run

nums = [1, 2, 3, 1]

seen={} → 1 not in seen → seen={1}

seen={1} → 2 not in seen → seen={1,2}

seen={1,2} → 3 not in seen → seen={1,2,3}

seen={1,2,3} → 1 IS in seen → return true ✓

Golang — Optimized (HashMap + Sort variant)

```
// Optimized — O(n) time, O(n) space

func containsDuplicate(nums []int) bool {

seen := make(map[int]struct{}, len(nums))

for _, num := range nums {

if _, exists := seen[num]; exists {

return true

}

seen[num] = struct{}{}

}

return false

}


// Alternative: Sort-based — O(n log n) time, O(1) space
```

```
import "sort"

func containsDuplicateSort(nums []int) bool {

sort.Ints(nums)

for i := 1; i &lt; len(nums); i++ {

if nums[i] == nums[i-1] {

return true

}

}

return false

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(n)**

Single pass through array. HashMap lookup and insert are O(1) average. Worst case: n distinct elements all stored in set.

■ 5. Pattern Explanation

■ Pattern
Pattern: HashSet for membership tracking. Whenever you need to answer 'have I seen this before?' in O(1), a HashSet is your tool.

When to use this pattern:

- Detecting duplicates in streams or arrays
- Checking if two arrays share common elements
- Finding the first repeated element
- Cycle detection in sequences

How to identify in unseen problems: If a problem asks 'has this appeared before?', 'is this unique?', 'find the first repeat' — HashSet is almost certainly the right tool. The key signal is membership queries on a growing collection.

■ 6. Edge Cases & Breaking Scenarios

- Empty array [] → loop never runs → returns false (correct)
- Single element [42] → no j to compare → returns false (correct)
- All same [5,5,5,5] → duplicate found on second element, early exit
- Very large range [-10^9, 10^9] → array indexing fails, HashMap handles it

- Sorted input [1,2,3,4,5] → no duplicates, full scan required

■ 7. Real-World Backend Connection

■ Real World
In PostgreSQL, UNIQUE constraints run a hash-based lookup on every INSERT. The B-Tree index structure allows O(log n) duplicate checks. Redis SADD command uses a HashSet — adding an existing member returns 0 (duplicate detected). Rate limiting systems use Redis Sets to track seen request IDs.

■■ 8. Common Mistakes

- Using a slice instead of a map for 'seen' — slice lookup is O(n), negating the optimization
- Comparing nums[i] == nums[i+1] without sorting first — only catches adjacent duplicates
- Off-by-one: inner loop starting at j=i (compares element with itself, always a 'duplicate')
- Not handling nil/empty input gracefully in production code

---

2 Valid Anagram

■ 1. Problem Understanding

Given two strings **s** and **t**, return **true** if **t** is an anagram of **s**, and **false** otherwise. An anagram is a word formed by rearranging the letters of another, using all original letters exactly once.

Real-World Analogy

Think of Scrabble tiles. You have a bag of tiles for the word 'LISTEN'. An anagram checker asks: 'Can I rearrange ALL these exact tiles to form SILENT?' Yes — same letters, same count, different order. Now try 'HELLO' → 'WORLD': impossible, different letters entirely. This is anagram checking.

What Problem Does This Solve?

- Search engines: grouping words that are anagrams for 'did you mean?' suggestions
- Plagiarism detection: identifying texts that rearrange words to avoid detection
- Cryptography: anagram ciphers are among the oldest encryption methods
- Grouping words in NLP (Natural Language Processing) pipelines

Input / Output

- s = 'anagram', t = 'nagaram' → true
- s = 'rat', t = 'car' → false
- s = 'ab', t = 'a' → false (different lengths)

■ 2. Constraints Analysis

- 1 ≤ s.length, t.length ≤ 5×10^4 → Up to 50,000 characters
- s and t consist of lowercase English letters → Only 26 distinct characters!
- This constraint is CRUCIAL: allows fixed-size array instead of hashmap
- Edge case: different lengths → immediately false, no need to compare
- Edge case: single character strings → compare directly

■ 3. Brute Force Approach

Intuition: Sort both strings. If they're anagrams, sorted versions must be identical. 'LISTEN' sorted → 'EILNST', 'SILENT' sorted → 'EILNST'. Equal? → Anagram!

Step-by-step

- Check if len(s) == len(t). If not, return false immediately
- Convert both strings to character slices
- Sort both slices
- Compare character by character — if all match, return true

Golang — Sort Approach (Brute Force)

```
// Sort-based approach — O(n log n)

import "sort"


func isAnagramSort(s string, t string) bool {

if len(s) != len(t) {

return false

}

sBytes := []byte(s)

tBytes := []byte(t)

sort.Slice(sBytes, func(i, j int) bool { return sBytes[i] &lt; sBytes[j] })

sort.Slice(tBytes, func(i, j int) bool { return tBytes[i] &lt; tBytes[j] })

return string(sBytes) == string(tBytes)

}
```

■ Time Complexity	■ Space Complexity
O(n log n)	**O(n)**

Sorting dominates. Allocating byte slices takes O(n) space.

■ 4. Optimized Approach — Frequency Count

Key Insight: Anagrams have identical character frequency distributions. Count how many times each character appears in s, then subtract counts for t. If all counts are zero at the end → anagram. Since only lowercase letters exist (a-z = 26), use a fixed-size array of 26 integers.

Step-by-step

- Return false if lengths differ
- Initialize count[26] = {0}
- For each char in s: count[char - 'a']++
- For each char in t: count[char - 'a']--
- If any count[i] != 0 → not anagram → return false
- Otherwise return true

Dry Run

s='rat', t='car'

After s: count[r]=1, count[a]=1, count[t]=1

After t: count[r]-=1→0, count[a]-=1→0, count[c]-=1→-1

count[c]=-1 ≠ 0 → return false ✓

Golang — Optimized (Frequency Count)

```
// Optimized — O(n) time, O(1) space (fixed 26-char array)

func isAnagram(s string, t string) bool {

if len(s) != len(t) {

return false

}

var count [26]int

for i := 0; i &lt; len(s); i++ {

count[s[i]-'a']++

count[t[i]-'a']--

}

for _, c := range count {

if c != 0 {

return false

}
```

```
}

return true

}


// Unicode-safe version (for non-ASCII characters)

func isAnagramUnicode(s string, t string) bool {

if len(s) != len(t) {

return false

}

freq := make(map[rune]int)

for _, ch := range s {

freq[ch]++

}

for _, ch := range t {

freq[ch]--

if freq[ch] &lt; 0 {

return false

}

}

return true

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Single pass, fixed-size array of 26 ints regardless of input size. If Unicode variant: O(k) space where k = unique characters.

■ 5. Pattern Explanation

■ **Pattern**
Pattern: Character Frequency Counting. A fixed-size frequency array is a constant-space HashMap when the character set is bounded. This pattern appears in: anagram grouping, minimum window substring, permutation in string.

The key insight is that equality of frequency distributions is equivalent to anagram equality. This transforms a string comparison problem into an arithmetic problem.

■ 6. Edge Cases & Breaking Scenarios

- Different lengths: 'ab' vs 'abc' → early return false, saves computation
- Empty strings: '' vs '' → true (both have zero frequency for all chars)
- Single char: 'a' vs 'b' → false
- Unicode: '■■' vs '■■' → need rune-based (UTF-8) approach
- Case sensitivity: 'Listen' vs 'Silent' → false unless we lowercase first

■ 7. Real-World Backend Connection

■ **Real World**
Elasticsearch uses character frequency analysis for fuzzy matching and 'did you mean?' features. Spell checkers use anagram indices: given a misspelled word, find all valid dictionary words that are anagrams. Database full-text search indexes often maintain term frequency vectors — the exact same data structure.

■■ 8. Common Mistakes

- Not checking length first — leads to false positives like 'ab' vs 'aab'
- Using byte indexing for Unicode strings in Go — rune conversion required
- Forgetting that 'a'-'a'=0, so array[char-'a'] maps a→0, b→1, ..., z→25
- Sorting mutable strings — in Go, strings are immutable, must convert to []byte

---

3 Two Sum

■ 1. Problem Understanding

Given an array of integers **nums** and an integer **target**, return the **indices** of the two numbers that add up to the target. You may assume exactly one solution exists, and you cannot use the same element twice.

Real-World Analogy

You're at a grocery store with a budget of exactly $20. You have a list of items with prices. You need to find exactly two items whose prices together total $20 — and you need to know WHICH items they are (their positions in the list), not just the prices. Two Sum is this exact problem, generalized to any numbers and any

target.

What Problem Does This Solve?

- Financial systems: finding two transactions that sum to a target amount (fraud detection)
- Recommendation systems: pairing complementary products that fit a budget
- Networking: finding two packet sizes that fill a buffer to exactly capacity
- Foundation for 3Sum, 4Sum, and k-Sum problems

Input / Output

- nums = [2, 7, 11, 15], target = 9 → [0, 1] (2+7=9)
- nums = [3, 2, 4], target = 6 → [1, 2] (2+4=6, NOT 3+3)
- nums = [3, 3], target = 6 → [0, 1] (different indices, same value)

■ 2. Constraints Analysis

- 2 ≤ nums.length ≤ 10^4 → At least two elements guaranteed
- -10^9 ≤ nums[i] ≤ 10^9 → Negative numbers are valid
- -10^9 ≤ target ≤ 10^9 → Target can be negative too
- Exactly one valid answer guaranteed → no need to handle 'no solution'
- Cannot use same element twice → two DIFFERENT indices required

■ 3. Brute Force Approach

Intuition: Check every possible pair. For each element nums[i], check if there's any nums[j] (j≠i) such that nums[i] + nums[j] == target.

Golang — Brute Force

```
// Brute Force — O(n²) time, O(1) space

func twoSumBrute(nums []int, target int) []int {

n := len(nums)

for i := 0; i &lt; n; i++ {

for j := i + 1; j &lt; n; j++ {

if nums[i]+nums[j] == target {

return []int{i, j}

}

}

}

return nil // guaranteed one solution exists, so unreachable
```

```
}
```

■ Time Complexity	■ Space Complexity
$O(n^2)$	$O(1)$

n(n-1)/2 pairs checked. For n=10,000: 50 million operations.*

■ 4. Optimized Approach — HashMap (Complement Lookup)

Key Insight: For each number x, we need to find its 'complement' = target - x. Instead of scanning the whole array for the complement, store previously seen numbers in a HashMap (value → index). Then for each new number, check if its complement is already in the map — O(1) lookup.

Step-by-step

- Create empty map: seen = {} (value → index)
- For each index i, value num in nums:

complement = target - num

If complement exists in seen → return [seen[complement], i]

Else add seen[num] = i

Dry Run

nums=[2,7,11,15], target=9

i=0, num=2: complement=7, not in seen → seen={2:0}

i=1, num=7: complement=2, 2 IS in seen → return [0, 1] ✓

Golang — Optimized (One-Pass HashMap)

```
// Optimized — O(n) time, O(n) space

func twoSum(nums []int, target int) []int {

seen := make(map[int]int) // value -> index

for i, num := range nums {

complement := target - num

if j, ok := seen[complement]; ok {

return []int{j, i}

}

seen[num] = i

}
```

```
return nil // unreachable given problem guarantees

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(n)**

Single pass. Each element: one HashMap lookup + one insert, both O(1) average. Space: at most n entries in the map.

■ 5. Pattern Explanation

■ Pattern

Pattern: Complement Search via HashMap. When you need 'find X such that X + Y = target', rearrange to 'find target - Y'. Store seen elements to check if complement was seen before. This pattern generalizes to: Two Sum II (sorted), 3Sum, 4Sum, Pair with Given Difference.

Mental Model: HashMap as a 'memory' of past elements. Instead of looking forward (is my complement ahead?), look backward (did I see my complement already?). This converts O(n) future searches into O(1) past lookups.

■ 6. Edge Cases & Breaking Scenarios

- Negative numbers: nums=[-3, 4, 7], target=1 → complement=-3+1=... (works fine)
- Same value twice: nums=[3,3], target=6 → must NOT store until AFTER checking complement
- Large values: nums=[10^9, -10^9], target=0 → integer overflow? In Go, int is 64-bit on modern systems, safe
- Zero: nums=[0, 4, 3, 0], target=0 → [0, 3] (two zeros)

■ 7. Real-World Backend Connection

■ Real World

Fraud detection systems use Two Sum logic: 'find two transactions in this account that sum to a suspicious amount.' Trading algorithms: 'find two assets whose combined exposure equals the risk budget.' Budget reconciliation: 'find two invoice line items that sum to an unaccounted-for amount.'

■■ 8. Common Mistakes

- Returning values instead of INDICES — problem explicitly asks for indices
- Not handling same-index reuse: if target=6, nums=[3,3] must use indices 0 AND 1
- Adding to map BEFORE checking complement — causes same-element reuse bug
- Assuming input is sorted — Two Sum does not guarantee sorted order

4 Group Anagrams

1. Problem Understanding

Given an array of strings **strs**, group the anagrams together. You can return the answer in any order. An anagram is a word formed by rearranging the letters of another.

Real-World Analogy

A librarian receives a pile of book titles. Some titles use the same letters in different order ('EVIL', 'VILE', 'LIVE'). The librarian needs to sort these into separate piles where every title in a pile is an anagram of each other. This is Group Anagrams — not just detecting anagrams, but clustering them.

What Problem Does This Solve?

- Search engines: grouping synonym clusters and query variations
- Word game solvers: finding all valid words for a given set of letters
- Data deduplication with semantic equivalence

Input / Output

- strs = ["eat","tea","tan","ate","nat","bat"]
- Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

2. Constraints Analysis

- 1 ≤ strs.length ≤ 10^4
- 0 ≤ strs[i].length ≤ 100
- strs[i] consists of lowercase letters only
- Empty string '' is a valid word — it's an anagram of itself

3. Brute Force Approach

Intuition: For each word, compare with every other word using isAnagram(). Group matching words together. $O(n^2 * k)$ where k is word length.

4. Optimized Approach — Sorted Key HashMap

Key Insight: All anagrams, when sorted, produce the SAME string. 'eat'→'aet', 'tea'→'aet', 'ate'→'aet'. Use this sorted string as a HashMap key. Group all original words under their sorted key.

Dry Run

"eat"→sort→"aet" → map["aet"]=["eat"]

"tea"→sort→"aet" → map["aet"]=["eat","tea"]

"tan"→sort→"ant" → map["ant"]=["tan"]

"ate"→sort→"aet" → map["aet"]=["eat","tea","ate"]

"nat"→sort→"ant" → map["ant"]=["tan","nat"]

"bat"→sort→"abt" → map["abt"]=["bat"]

Golang — Optimized (Sorted Key + Frequency Array Key)

```
// Optimized — O(n * k log k) using sorted string as key

import "sort"


func groupAnagrams(strs []string) [][]string {

groups := make(map[string][]string)

for _, s := range strs {

// Sort the string to create a canonical key

b := []byte(s)

sort.Slice(b, func(i, j int) bool { return b[i] &lt; b[j] })

key := string(b)

groups[key] = append(groups[key], s)

}

result := make([][]string, 0, len(groups))

for _, group := range groups {

result = append(result, group)

}

return result

}


// Even faster: O(n * k) using frequency count as key

func groupAnagramsFreq(strs []string) [][]string {

groups := make(map[[26]int][]string)

for _, s := range strs {

var count [26]int

for _, ch := range s {
```

```
count[ch-'a']++
}
groups[count] = append(groups[count], s)
}
result := make([][]string, 0, len(groups))
for _, group := range groups {
result = append(result, group)
}
return result
}
```

■ Time Complexity	■ Space Complexity
O(n * k log k)	**O(n * k)**

*n = number of strings, k = max string length. Sorting each string: O(k log k). Better: O(n*k) with frequency count key.*

■ 5. Pattern Explanation

■ Pattern

Pattern: Canonical Form + HashMap Grouping. Transform each element into a canonical (standard) form that all equivalent elements share. Use as HashMap key to group originals. Applies to: grouping equivalent fractions, grouping isomorphic strings, grouping equivalent trees.

■ 6. Edge Cases

- Single word: ["abc"] → [["abc"]]
- All same: ["aa","aa"] → [["aa","aa"]]
- Empty string: [""] → [[""]]
- No anagrams: ["abc","def"] → [["abc"],["def"]]

■ 7. Real-World Backend Connection

■ Real World

Search engines like Google use canonical forms to cluster semantically equivalent queries. 'buy shoes online' and 'online shoes buy' after stopword removal and sorting → same canonical form → same result cluster. Elasticsearch's token normalization pipeline does exactly this.

■■ 8. Common Mistakes

- Using the word itself as key (not sorted) — misses anagram grouping
- Trying to use []int as map key — in Go, arrays (fixed size) work as keys, slices don't
- Sorting without converting to []byte — strings are immutable in Go

---

5 Top K Frequent Elements

■ 1. Problem Understanding

Given an integer array **nums** and an integer **k**, return the **k most frequent elements**. You may return the answer in any order.

Real-World Analogy

A music streaming platform wants to display the Top 10 most-played songs this week. From millions of play events, it must count each song's plays and return the 10 with the highest counts. This is exactly Top K Frequent — count occurrences, return top K by count.

What Problem Does This Solve?

- Analytics dashboards: top K searched queries, top K error codes, top K URLs
- Recommendation engines: most purchased items, most watched videos
- Log analysis: which services throw the most errors
- Ad systems: most clicked ads

Input / Output

- nums = [1,1,1,2,2,3], k = 2 → [1, 2]
- nums = [1], k = 1 → [1]

■ 2. Constraints Analysis

- 1 ≤ nums.length ≤ 10^5
- -10^4 ≤ nums[i] ≤ 10^4
- k is in range [1, number of unique elements]
- The answer is guaranteed to be unique
- Must be better than O(n log n) — implies we need bucket sort or heap

■ 3. Brute Force Approach

Intuition: Count frequencies with HashMap, then sort by frequency descending, take first k.

```
Golang — Sort Approach
```

```go
// Sort-based — O(n log n)

import "sort"


func topKFrequentSort(nums []int, k int) []int {

freq := make(map[int]int)

for _, n := range nums {

freq[n]++

}

keys := make([]int, 0, len(freq))

for key := range freq {

keys = append(keys, key)

}

sort.Slice(keys, func(i, j int) bool {

return freq[keys[i]] > freq[keys[j]]

})

return keys[:k]

}
```

■ Time Complexity	■ Space Complexity
O(n log n)	**O(n)**
Sorting n unique elements dominates.	

■ 4. Optimized Approach — Bucket Sort

Key Insight: The maximum possible frequency of any element is n (if all elements are the same). Create an array of 'buckets' indexed by frequency (0 to n). Place each number in the bucket corresponding to its frequency. Then read buckets from high to low, collecting k elements.

Step-by-step

- Count frequency of each element → map[int]int
- Create buckets: [][]int of size n+1 (index = frequency)
- For each (value, freq) in freq map: buckets[freq] = append(buckets[freq], value)
- Iterate buckets from index n down to 0, collect until we have k elements

Golang — Bucket Sort O(n)

```
// Bucket Sort — O(n) time, O(n) space

func topKFrequent(nums []int, k int) []int {

// Step 1: Count frequencies

freq := make(map[int]int)

for _, n := range nums {

freq[n]++

}

// Step 2: Create frequency buckets (index = frequency)

buckets := make([][]int, len(nums)+1)

for num, count := range freq {

buckets[count] = append(buckets[count], num)

}

// Step 3: Collect top k from highest frequency down

result := make([]int, 0, k)

for i := len(buckets) - 1; i >= 0 && len(result) &lt; k; i-- {

result = append(result, buckets[i]...)

}

return result[:k]

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(n)**

Counting: O(n). Bucketing: O(n). Collection: O(n). No sorting! This beats the O(n log n) constraint requirement.

■ 5. Pattern Explanation

■ Pattern

Pattern: Bucket Sort for bounded frequency problems. When the range of values is bounded by n (the input size), bucket sort achieves O(n). Key insight: frequency is always between 1 and n, so we can use frequency as an index.

■ 6. Edge Cases

- k = n (all unique elements): return all elements
- All same element: one bucket at index n, return it
- k=1: return the single most frequent element
- Multiple elements with same frequency: any can be chosen (problem guarantees unique answer)

■ 7. Real-World Backend Connection

■ Real World
Redis's ZRANGEBYSCORE is built on sorted sets — essentially a Top-K structure. Apache Kafka metrics pipelines count message frequencies per topic and surface top K topics by volume. Nginx access logs analysis uses this exact pattern to find top K IPs by request count.

■■ 8. Common Mistakes

- Using a min-heap of size k: valid but O(n log k) — not as fast as bucket sort
- Bucket array size: must be len(nums)+1, not len(freq)+1
- Collecting too many elements if multiple elements share the k-th frequency

---

6 Product of Array Except Self

■ 1. Problem Understanding

Given an integer array **nums**, return an array **answer** such that **answer[i]** is equal to the product of all elements of nums EXCEPT nums[i]. You must solve it in O(n) time WITHOUT using division.

Real-World Analogy

Five machines on a production line, each contributing a multiplier to the final output. A quality engineer wants to know: 'If I remove machine i, what would the total output multiplier be?' They need this for ALL machines simultaneously, without dividing the total output by each machine's contribution (division might give precision errors or be undefined if a machine contributes 0).

Input / Output

- nums = [1,2,3,4] → [24,12,8,6] (24=2*3*4, 12=1*3*4, 8=1*2*4, 6=1*2*3)
- nums = [-1,1,0,-3,3] → [0,0,9,0,0]

■ 2. Constraints Analysis

- 2 ≤ nums.length ≤ 10^5
- -30 ≤ nums[i] ≤ 30 → Products can be very large

- No division allowed! This eliminates the obvious O(n) approach
- Zeros: if nums[i]=0, output[i] = product of all others (possibly 0 if multiple zeros)

■ 3. Brute Force Approach

Intuition: For each index i, multiply all elements except index i. Nested loops.

■ Time Complexity	■ Space Complexity
$O(n^2)$	$O(1)$
For each of n positions, multiply n-1 elements.	

■ 4. Optimized Approach — Prefix and Suffix Products

Key Insight: For each index i, the product of all elements except i equals: (product of all elements LEFT of i) × (product of all elements RIGHT of i). Precompute prefix products (left to right) and suffix products (right to left). Combine them.

Step-by-step with Dry Run

nums = [1, 2, 3, 4]

Prefix: [1, 1, 2, 6] (prefix[i] = product of nums[0..i-1])

prefix[0]=1 (nothing left), prefix[1]=1, prefix[2]=1*2=2, prefix[3]=2*3=6

Suffix: [24, 12, 4, 1] (suffix[i] = product of nums[i+1..n-1])

suffix[3]=1, suffix[2]=4, suffix[1]=3*4=12, suffix[0]=2*3*4=24

Answer[i] = prefix[i] * suffix[i]

[1*24, 1*12, 2*4, 6*1] = [24, 12, 8, 6] ✓

Golang — Optimized (O(1) extra space)

```
// O(n) time, O(1) extra space (output array doesn't count)

func productExceptSelf(nums []int) []int {

n := len(nums)

result := make([]int, n)


// First pass: fill result with prefix products

// result[i] = product of nums[0..i-1]

result[0] = 1

for i := 1; i &lt; n; i++ {
```

```
result[i] = result[i-1] * nums[i-1]

}


// Second pass: multiply by suffix products (right to left)

// suffix tracks product of nums[i+1..n-1]

suffix := 1

for i := n - 1; i >= 0; i-- {

result[i] *= suffix

suffix *= nums[i]

}


return result

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Two passes. Output array is O(n) but required by the problem. No extra auxiliary space.

■ 5. Pattern Explanation

■ Pattern

Pattern: Prefix/Suffix Precomputation. When you need a result that depends on elements 'before and after' a position, precompute prefix and suffix aggregates. Generalizes to: prefix sum, prefix max/min, range queries. Core insight: split the 'except self' into left contribution × right contribution.

■ 6. Edge Cases

- Single zero in array: all outputs = 0 except at the zero's index (= product of rest)
- Two or more zeros: all outputs = 0
- Negative numbers: handled naturally (prefix/suffix accumulate sign correctly)
- All ones [1,1,1]: output = [1,1,1]

■ 7. Real-World Backend Connection

■ **Real World**
Database query optimizers use this pattern: when computing aggregates (SUM, PRODUCT) over ranges excluding a specific row, they maintain running prefix and suffix aggregates. Signal processing: convolution excludes the center sample. Financial risk: portfolio sensitivity = product of all other asset volatilities.

■■ 8. Common Mistakes

- Using division: fails when any element is 0 (division by zero)
- Using total product / nums[i]: same division problem
- Off-by-one in prefix/suffix initialization
- Creating separate prefix AND suffix arrays: valid but O(n) extra space

---

7 Valid Sudoku

■ 1. Problem Understanding

Determine if a 9×9 Sudoku board is valid. The board is valid if: each row contains digits 1-9 without repetition, each column contains digits 1-9 without repetition, and each of the nine 3×3 sub-boxes contains digits 1-9 without repetition. Empty cells are marked with '.'.

Real-World Analogy

A Sudoku puzzle validator for a mobile app. Users fill in numbers, and after each move, the app checks 'is the current board still valid (no violations yet)?' — not 'is the puzzle solved?' This is board validation, not puzzle solving.

What Problem Does This Solve?

- Constraint satisfaction validation: checking multi-dimensional uniqueness constraints
- Database: validating composite uniqueness across multiple dimensions
- Grid-based game validation: chess, crossword, kakuro

■ 2. Constraints Analysis

- Board is always 9×9 (fixed size) → O(1) time technically!
- Each cell contains '1'-'9' or '.'
- Need to validate: 9 rows + 9 columns + 9 boxes = 27 constraint groups
- Key: identify which 3×3 box a cell (r,c) belongs to → box index = (r/3)*3 + (c/3)

■ 3. Brute Force Approach

Intuition: Separate passes for rows, columns, boxes. For each row: check no duplicates. For each column: check no duplicates. For each box: check no duplicates. This is O(81*3) = O(1) for fixed board.

■ 4. Optimized Approach — Single Pass with 3 HashSets

Key Insight: Instead of 3 separate passes, do ONE pass through the board. For each cell, simultaneously check its row, column, AND box constraint. Use separate sets for rows[9], cols[9], boxes[9].

Box index formula: **boxIdx = (row/3)*3 + (col/3)**

For row=4, col=7: (4/3)*3 + (7/3) = 1*3+2 = 5 → box 5 (center-right)

Golang — Single Pass Validation

```
// Single pass — O(1) time and space (board is always 9x9)

func isValidSudoku(board [][]byte) bool {

// Use sets for each row, column, and 3x3 box

rows := [9]map[byte]bool{}

cols := [9]map[byte]bool{}

boxes := [9]map[byte]bool{}


for i := range rows {

rows[i] = make(map[byte]bool)

cols[i] = make(map[byte]bool)

boxes[i] = make(map[byte]bool)

}


for r := 0; r &lt; 9; r++ {

for c := 0; c &lt; 9; c++ {

val := board[r][c]

if val == '.' {

continue

}

// Determine box index

boxIdx := (r/3)*3 + (c/3)


// Check all three constraints simultaneously

if rows[r][val] || cols[c][val] || boxes[boxIdx][val] {
```

```
return false // duplicate found

}

rows[r][val] = true

cols[c][val] = true

boxes[boxIdx][val] = true

}

}

return true

}
```

■ Time Complexity	■ Space Complexity
O(1)	**O(1)**

Board is always 9×9 = 81 cells. Fixed computation regardless of 'input size'. Technically O(81) = O(1).

■ 5. Pattern Explanation

■ Pattern

Pattern: Multi-dimensional constraint checking with parallel HashSets. When validating uniqueness across multiple groupings simultaneously, maintain one set per group type and check/update all groups in a single pass. The box index formula (r/3)*3 + (c/3) is a key trick for 2D grid partitioning.

■ 6. Edge Cases

- All '.' (empty board): valid (no violations)
- Duplicate in first row: caught immediately
- Valid partial board: only filled cells are checked
- Same number in box but different row/col: caught by box set

■ 7. Real-World Backend Connection

■ Real World

Multi-dimensional uniqueness constraints appear in: database composite unique indexes (unique across multiple columns), scheduling systems (no two meetings in same room at same time), network routing tables (no duplicate routes per subnet per VRF). The parallel-sets approach directly maps to multi-column unique constraint validation.

■■ 8. Common Mistakes

- Wrong box index formula: common errors are (r%3)*3+(c/3) or wrong combinations
- Not skipping '.' cells — '.' compared to '.' would incorrectly flag a 'duplicate'
- Checking if board is SOLVED vs VALID — problem asks for valid, not solved

8 Encode and Decode Strings

1. Problem Understanding

Design an algorithm to encode a list of strings to a single string and decode it back. The encoded string is sent over the network and decoded on the other side. Strings can contain any character including the delimiter itself.

Real-World Analogy

Shipping department packing multiple items in one box. They need a packing slip that tells the unpacker exactly how to separate the items. The challenge: item names might contain the separator character (e.g., if you use '|' as separator, what happens when an item is named 'Salt|Pepper'?). You need a protocol that handles ANY content — this is serialization.

What Problem Does This Solve?

- Network protocols: encoding variable-length messages without ambiguous delimiters
- Database serialization: storing arrays in single columns
- Inter-process communication: passing structured data through pipes
- RPC frameworks: serializing function arguments

2. Constraints Analysis

- $0 \leq$ strs.length ≤ 200
- $0 \leq$ strs[i].length ≤ 200
- strs[i] contains ANY character (256 ASCII values)
- Simple delimiter approach FAILS if delimiter appears in strings
- Need length-prefix encoding for robustness

3. Brute Force — Why It Fails

Naive approach: Join with a special delimiter like '|'. Fails immediately for strs=['a|b', 'c'] → encoded='a|b|c' → decoded=['a','b','c'] — WRONG! We have 3 strings instead of 2.

4. Optimized Approach — Length-Prefix Encoding

Key Insight: Prefix each string with its length followed by a separator. Format: '#<string>'. Since length is explicit, we know EXACTLY how many bytes to read, regardless of content. Even if '#' appears in the string, we skip exactly 'length' bytes.

Encoding Example

["hello","world"] → "5#hello5#world"

["a#b","c"] → "3#a#b1#c" (reads 3 bytes after first #, so a#b is one string)

Decoding Algorithm

i=0: read digits until '#' → length=5, read 5 chars → 'hello', i=7

i=7: read digits until '#' → length=5, read 5 chars → 'world', done

Golang — Length-Prefix Encode/Decode

```
// Length-prefix encoding: "<len>#&lt;string>"

import (

"fmt"

"strconv"

"strings"

)


// Encodes a list of strings to a single string

func encode(strs []string) string {

var sb strings.Builder

for _, s := range strs {

sb.WriteString(fmt.Sprintf("%d#%s", len(s), s))

}

return sb.String()

}


// Decodes a single string to a list of strings

func decode(s string) []string {

result := []string{}

i := 0

for i &lt; len(s) {

// Find the '#' delimiter

j := i
```

```
for s[j] != '#' {

j++

}

// Parse length

length, _ := strconv.Atoi(s[i:j])

// Extract string of that exact length

result = append(result, s[j+1:j+1+length])

i = j + 1 + length

}

return result

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(n)**

n = total characters across all strings. Encoding: one pass. Decoding: one pass.

■ 5. Pattern Explanation

■ Pattern

Pattern: Length-Prefix Framing (also called 'LV encoding' — Length-Value). This is the foundational protocol used in: HTTP/2 binary framing, Protocol Buffers (protobuf), MessagePack, TLV (Type-Length-Value) in network protocols. The key: encode metadata (length) before content, so the decoder knows boundaries without ambiguous delimiters.

■ 6. Edge Cases

- Empty list [] → encode='', decode=[]
- List with empty string ['] → encode='0#', decode=['']
- Strings with '#' → length prefix prevents misinterpretation
- Strings with only '#' ['###'] → '3####', decode reads 3 bytes → '###' ✓
- Very long strings → length prefix as string of digits handles any length

■ 7. Real-World Backend Connection

■ **Real World**
Protocol Buffers (protobuf) uses varint-prefixed fields. Redis RESP protocol uses length-prefixed bulk strings: '$6\r\nfoobar\r\n'. HTTP Content-Length header is the same concept. gRPC frames messages with 5-byte headers (1 compression flag + 4-byte length). Every modern binary protocol uses this pattern.

■■ 8. Common Mistakes

- Using '\0' (null byte) as delimiter — fails for binary strings
- Assuming '#' won't appear in strings — always wrong assumption
- Using JSON encoding — technically works but adds overhead; misses the point of the problem
- Forgetting to handle empty strings (length=0)

---

9 Longest Consecutive Sequence

■ 1. Problem Understanding

Given an unsorted array of integers **nums**, return the length of the longest consecutive elements sequence. You must write an algorithm that runs in **O(n)** time.

Real-World Analogy

A company has employee IDs assigned randomly over the years: [100, 4, 200, 1, 3, 2]. They want to find the longest 'streak' of consecutive IDs (1, 2, 3, 4 is a streak of length 4). This tells them the longest range of IDs that have been continuously assigned, which helps in ID management.

Input / Output

- nums = [100,4,200,1,3,2] → 4 (sequence: 1,2,3,4)
- nums = [0,3,7,2,5,8,4,6,0,1] → 9 (0,1,2,3,4,5,6,7,8)

■ 2. Constraints Analysis

- 0 ≤ nums.length ≤ 10^5
- -10^9 ≤ nums[i] ≤ 10^9 → Wide range, no direct array indexing
- MUST be O(n) → cannot sort (O(n log n))
- May contain duplicates → HashSet deduplicates automatically

■ 3. Brute Force — Sort-based

Approach: Sort the array, then scan for consecutive sequences. O(n log n) — violates constraint.

■ 4. Optimized Approach — HashSet + Sequence Start Detection

Key Insight: Load all numbers into a HashSet. For each number n, only start counting a sequence if (n-1) is NOT in the set — meaning n is the START of a sequence. Then count how far the sequence extends (n+1, n+2, ...). This ensures each number is visited at most twice.

Why This is O(n) — Critical Analysis

Each number is either a sequence start (visited once in outer loop + traversed in inner loop) or a non-start (skipped in outer loop). A number that is traversed in an inner loop is NOT a sequence start, so it won't trigger another inner loop. Total inner loop iterations across ALL outer iterations ≤ n.

Dry Run

nums=[100,4,200,1,3,2], set={100,4,200,1,3,2}

n=100: 99 not in set → start! 100∈set, 101∉set → len=1

n=4: 3 IS in set → skip

n=200: 199 not in set → start! 200∈set, 201∉set → len=1

n=1: 0 not in set → start! 1,2,3,4∈set, 5∉set → len=4 ✓

n=3: 2 IS in set → skip

n=2: 1 IS in set → skip

Max = 4

Golang — HashSet Sequence Detection

```
// O(n) time, O(n) space

func longestConsecutive(nums []int) int {

if len(nums) == 0 {

return 0

}

// Load into HashSet for O(1) lookup

numSet := make(map[int]bool, len(nums))

for _, n := range nums {

numSet[n] = true

}


maxLen := 0

for n := range numSet {

// Only start counting from sequence beginnings
```

```
if !numSet[n-1] {

length := 1

for numSet[n+length] {

length++

}

if length > maxLen {

maxLen = length

}

}

}

return maxLen

}
```

Time Complexity	Space Complexity
O(n)	O(n)

Each element visited at most twice: once for set construction, once during sequence traversal. Total inner loop iterations bounded by n.

5. Pattern Explanation

Pattern

Pattern: HashSet with selective starting points. When asked to find longest runs/streaks in O(n), use a set for O(1) membership, then only START counting from elements with no predecessor. This avoids redundant counting. Applies to: longest streak in binary data, finding maximal arithmetic progressions.

6. Edge Cases

- Empty array → return 0
- All same elements [3,3,3] → set={3}, longest=1
- Already consecutive [1,2,3,4] → one sequence start (1), length=4
- Negative numbers [-3,-2,-1,0,1] → works, sequence of 5
- Single element → return 1

7. Real-World Backend Connection

■ **Real World**
Database index analysis: finding the longest range of consecutive primary keys to detect fragmentation. Network IP allocation: finding the longest block of consecutive available IPs. Git commit graph: finding longest linear chain of commits. Time-series analysis: longest streak of daily active users.

■■ 8. Common Mistakes

- Sorting first — O(n log n), violates problem constraint
- Not checking for sequence start — causes $O(n^2)$ by restarting from middle of sequences
- Using original array instead of set — O(n) membership check instead of O(1)
- Not handling duplicates — set automatically deduplicates

10 Best Time to Buy & Sell Stock

■ 1. Problem Understanding

Given an array **prices** where prices[i] is the price on day i, find the maximum profit from a single buy-sell transaction. You must buy BEFORE you sell. Return 0 if no profit is possible.

Real-World Analogy

A stock trader has price data for the past year. They can make one purchase and one sale. When should they have bought and when should they have sold to maximize profit? They can't time-travel (must buy before sell). This is the fundamental stock optimization question.

Input / Output

- prices = [7,1,5,3,6,4] → 5 (buy day 2 @1, sell day 5 @6, profit=5)
- prices = [7,6,4,3,1] → 0 (prices always decrease, no profit possible)

■ 2. Constraints Analysis

- 1 ≤ prices.length ≤ 10^5
- 0 ≤ prices[i] ≤ 10^4
- Must buy before sell (temporal constraint)
- Single transaction only
- Return 0 if no profitable transaction exists

■ 3. Brute Force Approach

Intuition: Try every (buy_day, sell_day) pair where buy_day < sell_day. Track maximum profit.

■ **Time Complexity** ■ **Space Complexity**

O(n²)	**O(1)**
n(n-1)/2 pairs. For n=100,000: 5 billion operations.*	

■ 4. Optimized Approach — Sliding Window / One Pass

Key Insight: Track the minimum price seen so far (best buy price). For each day, compute profit if we sell today (current_price - min_price). Track maximum profit. The minimum must have occurred BEFORE the current day (guaranteed by left-to-right scan).

Dry Run

prices=[7,1,5,3,6,4]

day 0: price=7, minPrice=7, profit=0, maxProfit=0

day 1: price=1, minPrice=1, profit=0, maxProfit=0

day 2: price=5, minPrice=1, profit=4, maxProfit=4

day 3: price=3, minPrice=1, profit=2, maxProfit=4

day 4: price=6, minPrice=1, profit=5, maxProfit=5 ✓

day 5: price=4, minPrice=1, profit=3, maxProfit=5

Golang — One Pass Optimal

```
// One pass — O(n) time, O(1) space

func maxProfit(prices []int) int {

if len(prices) == 0 {

return 0

}

minPrice := prices[0]

maxProfit := 0


for _, price := range prices {

if price &lt; minPrice {

minPrice = price // Found a better buy day

} else if price-minPrice > maxProfit {

maxProfit = price - minPrice // Found better profit

}
```

```
}

return maxProfit

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Single scan, two integer variables. Classic example of transforming $O(n^2)$ to $O(n)$ by tracking running minimum.

■ 5. Pattern Explanation

■ Pattern

Pattern: Running minimum + greedy profit tracking. When optimizing buy-low-sell-high with temporal ordering, track the minimum seen so far (best buy) and compute the potential profit at each sell point. This is a special case of the sliding window pattern where the window's left boundary is the minimum index.

■ 6. Edge Cases

- Single day [5] → can't sell → return 0
- Descending prices [5,4,3,2,1] → no profit → return 0
- All same prices [3,3,3] → profit=0
- prices=[1,2] → buy day 0, sell day 1, profit=1
- Minimum at the very end → no profit (sell can't be before buy)

■ 7. Real-World Backend Connection

■ Real World

High-frequency trading systems use this exact algorithm on tick-by-tick price streams: maintain a running minimum and compute the 'paper profit' at each tick. Cryptocurrency arbitrage bots use variants of this across exchange pairs. Cloud cost optimization: find the cheapest spot instance price in a time window before a compute job starts.

■■ 8. Common Mistakes

- Using max - min without enforcing temporal order (sell must be after buy)
- Returning negative profit (not checking if profit > 0)
- Updating minPrice and profit simultaneously in wrong order
- Not initializing maxProfit = 0 (problem says return 0 if no profit)

11 Container With Most Water

1. Problem Understanding

Given n vertical lines represented by height[i], find two lines that together with the x-axis forms a container that can hold the most water. The container's water volume = min(height[left], height[right]) × (right - left).

Real-World Analogy

You have a row of walls of different heights. You want to place two walls as endpoints to create a trough that holds the most water. The water level is limited by the shorter wall. You want to maximize: shorter_wall_height × distance_between_walls.

Input / Output

- height = [1,8,6,2,5,4,8,3,7] → 49 (walls at index 1 and 8: min(8,7)×7=49)
- height = [1,1] → 1

2. Constraints Analysis

- n ≥ 2 → At least two walls
- 0 ≤ height[i] ≤ 10^4
- Water = min(h[l], h[r]) * (r-l)
- Brute force: $O(n^2)$ — try all pairs. Need O(n).

3. Brute Force

Intuition: Check every pair (i,j). Compute water, track maximum. $O(n^2)$.

4. Optimized Approach — Two Pointers

Key Insight: Start with pointers at both ends (maximum width). Moving a pointer inward reduces width. To potentially increase water, we MUST move the shorter wall's pointer (moving the taller one can never help — water is already capped by the shorter one). This greedy insight enables O(n).

Proof of Greedy Correctness

At pointers l,r: water = min(h[l],h[r])*(r-l). If we move the taller pointer (say h[r]>h[l]): new water ≤ min(h[l], h[r-1]) * (r-l-1) ≤ h[l] * (r-l) = current water (since h[l] was the bottleneck). So moving the taller pointer can never improve. Must move the shorter one.

Dry Run

height=[1,8,6,2,5,4,8,3,7]

l=0,r=8: min(1,7)*8=8, h[0]<h[8] → move l

l=1,r=8: min(8,7)*7=49, h[8]<h[1] → move r, maxWater=49

l=1,r=7: min(8,3)*6=18, h[7]<h[1] → move r

... eventually l meets r → done, return 49

Golang — Two Pointer Optimal

```
// Two Pointers — O(n) time, O(1) space

func maxArea(height []int) int {

left, right := 0, len(height)-1

maxWater := 0


for left &lt; right {

// Water is limited by shorter wall

h := min(height[left], height[right])

water := h * (right - left)

if water > maxWater {

maxWater = water

}

// Move the pointer with the shorter wall

if height[left] &lt; height[right] {

left++

} else {

right--

}

}

return maxWater

}


func min(a, b int) int {

if a &lt; b { return a }

return b

}
```

■ Time Complexity	■ Space Complexity
`O(n)`	`O(1)`

Pointers start at ends, meet in middle. Total moves = n-1. Each move is O(1).

■ 5. Pattern Explanation

■ Pattern

Pattern: Two Pointers shrinking from both ends. Used when: (1) searching for optimal pair in array, (2) one dimension (width) strictly decreases as pointers converge, (3) greedy insight tells you which pointer to move. Similar problems: Trapping Rain Water, 3Sum.

■ 6. Edge Cases

- [1,1] → min(1,1)*1 = 1
- All same height [5,5,5,5] → 5*(n-1)
- Monotonically increasing: greedy always moves left pointer
- One zero height: that wall contributes 0 to any container → immediately move it

■ 7. Real-World Backend Connection

■ Real World

Resource allocation: maximizing throughput between two rate-limited services (bottleneck = slower service). Network flow: finding the maximum data that can flow between two endpoints (limited by the minimum bandwidth link). Storage tiering: choosing two cache boundaries to maximize the data reuse window.

■■ 8. Common Mistakes

- Moving the taller pointer (fails to explore potentially better options)
- Forgetting that width decreases by 1 each step
- Using height[l]*height[r] instead of min(height[l],height[r]) * width
- Not initializing maxWater=0 before the loop

---

12 3Sum

■ 1. Problem Understanding

Given an integer array **nums**, return all unique triplets [nums[i], nums[j], nums[k]] such that i ≠ j ≠ k and nums[i] + nums[j] + nums[k] == 0. The solution set must not contain duplicate triplets.

Real-World Analogy

A financial auditor is looking for three transactions that perfectly cancel each other out (sum to zero): one credit, two debits of equal combined value. They need ALL such groups, without counting the same three transactions multiple times regardless of order.

Input / Output

- nums = [-1,0,1,2,-1,-4] → [[-1,-1,2],[-1,0,1]]
- nums = [0,1,1] → []
- nums = [0,0,0] → [[0,0,0]]

■ 2. Constraints Analysis

- 3 ≤ nums.length ≤ 3000 → $O(n^2)$ acceptable, $O(n^3)$ is not
- -10^5 ≤ nums[i] ≤ 10^5
- No duplicate triplets → sorting + careful pointer skipping handles this
- Triplet [a,b,c] with a≤b≤c considered same as [c,b,a]

■ 3. Brute Force

Approach: Three nested loops, check all triplets. $O(n^3)$ = 27 billion for n=3000. Need deduplication via Set. Completely infeasible.

■ 4. Optimized — Sort + Two Pointers

Key Insight: Sort the array. Fix one element (outerloop at index i). Then use Two Pointers (l=i+1, r=end) to find pairs that sum to -nums[i]. Sorting enables: (1) efficient two-pointer search, (2) easy duplicate skipping.

Step-by-step

- Sort nums
- For i from 0 to n-3:

Skip if nums[i] > 0 (sorted array, all future elements also > 0, sum > 0)

Skip if i > 0 and nums[i] == nums[i-1] (duplicate fixed element)

l=i+1, r=n-1, find pairs summing to -nums[i]:

If sum < target: l++

If sum > target: r--

If sum == target: add triplet, skip duplicates at both l and r

Golang — Sort + Two Pointers

```
// Sort + Two Pointers — O(n²) time, O(1) extra space

import "sort"


func threeSum(nums []int) [][]int {
```

```
sort.Ints(nums)

result := [][]int{}

n := len(nums)


for i := 0; i &lt; n-2; i++ {

// Optimization: all remaining elements are positive, can't sum to 0

if nums[i] > 0 {

break

}

// Skip duplicate values for the fixed element

if i > 0 && nums[i] == nums[i-1] {

continue

}


l, r := i+1, n-1

target := -nums[i]


for l &lt; r {

sum := nums[l] + nums[r]

if sum == target {

result = append(result, []int{nums[i], nums[l], nums[r]})

// Skip duplicates at both pointers

for l &lt; r && nums[l] == nums[l+1] { l++ }

for l &lt; r && nums[r] == nums[r-1] { r-- }

l++

r--

} else if sum &lt; target {

l++

} else {

r--
```

```
}
}
}
return result
}
```

■ Time Complexity	■ Space Complexity
$O(n^2)$	$O(1)$

Sort: O(n log n). Outer loop: O(n). Inner two-pointer: O(n). Total: O(n²). Space: O(1) extra (output doesn't count).

■ 5. Pattern Explanation

■ Pattern

Pattern: Fix-one + Two Pointers. For k-Sum problems, fix k-2 elements in nested loops, solve the remaining 2-Sum with two pointers. 3Sum = fix 1 + 2-pointer. 4Sum = fix 2 + 2-pointer. Sorting is required to enable two pointers and easy deduplication.

■ 6. Edge Cases

- All zeros [0,0,0,0] → one triplet [0,0,0]
- All same positives [1,1,1] → no triplet (can't sum to 0)
- nums[i] > 0 after sorting → break early (all sums positive)
- Duplicates: [-1,-1,-1,0,1,2] → must skip duplicate fixed elements

■ 7. Real-World Backend Connection

■ Real World

Transaction reconciliation: finding three transactions that net to zero (debits and credits balancing). Chemistry: finding three compounds that neutralize each other. Graph theory: finding 3-cycles. Physics simulation: finding three forces in equilibrium.

■■ 8. Common Mistakes

- Not sorting first — two pointers require sorted order
- Duplicate handling: skipping AFTER adding to result, not before
- i going to n-1 instead of n-2 (l and r need at least 2 elements)
- Not skipping duplicates for both l and r pointers

13 Trapping Rain Water

1. Problem Understanding

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining. Water is trapped between taller bars; the amount at each position is determined by the minimum of the maximum heights to its left and right, minus the current height.

Real-World Analogy

Imagine a mountain terrain cross-section viewed from the side. After heavy rain, water fills the valleys between peaks. A valley holds water up to the height of the shorter of the two surrounding peaks. A civil engineer calculating drainage capacity, or a city planner modeling flood zones, solves this exact problem.

Key Formula

water[i] = max(0, min(maxLeft[i], maxRight[i]) - height[i])

Input / Output

- height = [0,1,0,2,1,0,1,3,2,1,2,1] → 6
- height = [4,2,0,3,2,5] → 9

2. Constraints Analysis

- n ≥ 1, height[i] ≥ 0
- Bars at edges can never trap water (no wall on one side)
- At each position: water trapped = min(maxLeft, maxRight) - height (if positive)

3. Brute Force — O(n²)

Approach: For each index i, scan left for maxLeft and scan right for maxRight. Compute water[i]. Total: O(n²).

4. Optimized Approach 1 — Prefix/Suffix Arrays O(n)

Precompute: maxLeft[i] = max height from 0 to i. maxRight[i] = max height from i to n-1. Then water[i] = max(0, min(maxLeft[i], maxRight[i]) - height[i]). Two passes to build arrays, one pass to compute.

4b. Optimized Approach 2 — Two Pointers O(1) Space

Key Insight: Use left and right pointers moving inward. Track leftMax and rightMax. If leftMax ≤ rightMax, the water at left is determined by leftMax (regardless of what's on the right — rightMax is already guaranteed to be ≥ leftMax). Compute water and move left pointer. Symmetric for right.

```
Golang — Two Pointer Optimal O(1) Space
```

```
// Two Pointers — O(n) time, O(1) space

func trap(height []int) int {

if len(height) == 0 {

return 0

}

left, right := 0, len(height)-1

leftMax, rightMax := height[left], height[right]

water := 0


for left &lt; right {

if leftMax &lt;= rightMax {

left++

if height[left] > leftMax {

leftMax = height[left]

} else {

water += leftMax - height[left]

}

} else {

right--

if height[right] > rightMax {

rightMax = height[right]

} else {

water += rightMax - height[right]

}

}

}

return water

}
```

■ Time Complexity	■ Space Complexity
`O(n)`	`O(1)`

Single pass with two pointers. Prefix/suffix version also O(n) but uses O(n) space for arrays.

■ 5. Pattern Explanation

■ Pattern

Pattern: Two Pointers with monotonic tracking. The key insight: if leftMax ≤ rightMax, the left position's water is guaranteed by leftMax alone — we don't need to know the exact right boundary. This eliminates the need to precompute suffix arrays. Same pattern as Container With Most Water but computing sum instead of max.

■ 6. Edge Cases

- Flat ground [0,0,0] → 0 water
- No valleys [1,2,3,4] → 0 water (monotonically increasing)
- [3,0,3] → 3 water trapped in valley
- Single bar [5] → 0 water
- All same height [4,4,4] → 0 water

■ 7. Real-World Backend Connection

■ Real World

Urban flood modeling: computing water accumulation in terrain. Memory fragmentation analysis: computing 'wasted gaps' between allocated memory blocks (the gaps between high watermarks). Histogram operations: computing areas in histograms. Network buffer analysis: computing available space between high-tide marks in a ring buffer.

■■ 8. Common Mistakes

- Using height[i] as local max instead of running maxLeft/maxRight
- Not handling negative values from min-height being negative
- Forgetting bars at index 0 and n-1 can't trap water (no wall on one side)
- Two-pointer: moving wrong pointer (must move the side with smaller max)

■ STRINGS

Problems 14–21 · Sliding Window · Two Pointers · Dynamic Programming

14 Valid Palindrome

■ 1. Problem Understanding

A phrase is a palindrome if, after converting all uppercase to lowercase and removing all non-alphanumeric characters, it reads the same forward and backward. Given a string s, return true if it is a palindrome.

Real-World Analogy

DNA sequence analysis: some DNA segments read the same on both strands. Linguistic analysis: detecting palindromic phrases like 'A man, a plan, a canal: Panama'. Form validation: palindrome-based CAPTCHA verification. The key challenge is handling 'noise' characters (spaces, punctuation) before comparison.

Input / Output

- "A man, a plan, a canal: Panama" → true
- "race a car" → false
- " " → true (empty after cleaning = palindrome)

■ 2. Constraints Analysis

- 1 ≤ s.length ≤ 2×10^5
- s consists of printable ASCII characters
- Only alphanumeric characters count; case-insensitive
- Empty string after filtering → true (empty string is trivially palindrome)

■ 3. Brute Force — Clean then Compare

Approach: Filter only alphanumeric chars, lowercase all, compare with its reverse. Clean and simple but uses O(n) extra space.

Golang — Clean then Compare

```
// Clean + Reverse — O(n) time, O(n) space

import "unicode"


func isPalindromeClean(s string) bool {
```

```
cleaned := []rune{}

for _, ch := range s {

if unicode.IsLetter(ch) || unicode.IsDigit(ch) {

cleaned = append(cleaned, unicode.ToLower(ch))

}

}

n := len(cleaned)

for i := 0; i &lt; n/2; i++ {

if cleaned[i] != cleaned[n-1-i] {

return false

}

}

return true

}
```

■ 4. Optimized — Two Pointers In-Place

Key Insight: No need to create a cleaned copy. Use two pointers from both ends, skipping non-alphanumeric characters. Compare characters at valid positions.

Golang — Two Pointer O(1) Space

```
// Two Pointers — O(n) time, O(1) space

import "unicode"


func isPalindrome(s string) bool {

left, right := 0, len(s)-1

for left &lt; right {

// Skip non-alphanumeric from left

for left &lt; right && !isAlphaNum(rune(s[left])) {

left++

}
```

```
// Skip non-alphanumeric from right

for left &lt; right && !isAlphaNum(rune(s[right])) {

right--

}

// Compare (case-insensitive)

if unicode.ToLower(rune(s[left])) != unicode.ToLower(rune(s[right])) {

return false

}

left++

right--

}

return true

}


func isAlphaNum(ch rune) bool {

return unicode.IsLetter(ch) || unicode.IsDigit(ch)

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Each character visited at most once by each pointer. No auxiliary storage.

■ 5. Pattern Explanation

■ Pattern

Pattern: Two Pointers from both ends for palindrome checking. Whenever you need to verify symmetry without reversing, use inward-moving pointers. The skip-invalid-chars technique generalizes: 'filtered two pointers' — skip elements that don't meet criteria before comparing.

■ 6. Edge Cases

- " " (spaces only) → empty after filter → true
- "a" → single char → true
- "0P" → not palindrome

- All special chars → empty after filter → true
- Unicode: accented letters are still letters (unicode.IsLetter)

■ 7. Real-World Backend Connection

■ Real World
DNA reverse complement checking (palindromic sequences in molecular biology). Symmetric data structure validation. URL shorteners that generate palindromic codes for aesthetics. Input sanitization pipelines: validate normalized forms of user input.

■■ 8. Common Mistakes

- Not handling case: 'Aa' should be palindrome
- Not skipping non-alphanumeric: 'A man' ≠ 'namA' without filtering
- Using bytes instead of runes for Unicode strings in Go
- Off-by-one: loop condition left < right (not ≤) — equal means single center char

15 Longest Substring Without Repeating Characters

■ 1. Problem Understanding

Given a string s, find the length of the longest substring without repeating characters. A substring is a contiguous sequence of characters within the string.

Real-World Analogy

A typist practicing unique keystrokes: they want to find the longest run where they didn't press the same key twice consecutively. Or: a streaming cipher that generates unique character sequences — the security analyst wants to find the longest non-repeating window (no repeated characters = harder to crack by frequency analysis).

Input / Output

- "abcabcbb" → 3 ("abc")
- "bbbbb" → 1 ("b")
- "pwwkew" → 3 ("wke")

■ 2. Constraints Analysis

- 0 ≤ s.length ≤ 5×10^4
- s consists of English letters, digits, symbols and spaces
- 128 ASCII characters total → window can be at most 128 chars long

■ 3. Brute Force — $O(n^2)$ or $O(n^3)$

Approach: For every starting index i, expand right while characters are unique. $O(n^2)$ with HashSet, $O(n^3)$ with nested loops.

■ 4. Optimized — Sliding Window with HashMap

Key Insight: Maintain a sliding window [left, right]. Expand right by adding characters. When a duplicate is found at right, shrink left to just past the previous occurrence of that character (direct jump using stored index). This avoids redundant character checking.

Dry Run

s = "abcabcbb"

r=0: 'a', map={a:0}, left=0, len=1

r=1: 'b', map={a:0,b:1}, left=0, len=2

r=2: 'c', map={a:0,b:1,c:2}, left=0, len=3, maxLen=3

r=3: 'a', a was at 0 ≥ left=0 → left=1, map[a]=3, len=3

r=4: 'b', b was at 1 ≥ left=1 → left=2, map[b]=4, len=3

r=5: 'c', c was at 2 ≥ left=2 → left=3, map[c]=5, len=3

r=6: 'b', b was at 4 ≥ left=3 → left=5, map[b]=6, len=2

r=7: 'b', b was at 6 ≥ left=5 → left=7, len=1

Result: maxLen=3 ✓

Golang — Sliding Window Optimal

```
// Sliding Window — O(n) time, O(min(m,n)) space

// where m = size of character set (128 for ASCII)

func lengthOfLongestSubstring(s string) int {

charIndex := make(map[byte]int) // char -> last seen index

maxLen := 0

left := 0


for right := 0; right &lt; len(s); right++ {

ch := s[right]

// If char was seen and its last occurrence is within the window

if idx, ok := charIndex[ch]; ok && idx >= left {
```

```
left = idx + 1 // Shrink window past the duplicate

}

charIndex[ch] = right

if right-left+1 > maxLen {

maxLen = right - left + 1

}

}

return maxLen

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(min(m,n))**

Right pointer moves n steps total. Left pointer only moves forward. Each char processed at most twice. Space: map stores at most min(m,n) entries where m=charset size.

■ 5. Pattern Explanation

■ Pattern

Pattern: Sliding Window with HashMap for constraint tracking. The sliding window expands right greedily and contracts left only when a constraint is violated. Storing the LAST INDEX (not just presence) enables direct jumping of the left boundary — O(n) instead of O(n^2). Applies to: minimum window substring, permutation in string, max consecutive ones.

■ 6. Edge Cases

- Empty string "" → 0
- "a" → 1
- All same "aaaa" → 1
- All unique "abcd" → 4 (whole string)
- Unicode: need rune-based map for multi-byte characters

■ 7. Real-World Backend Connection

■ Real World
Session token generation: longest unique character sequence for maximum entropy. Network packet analysis: longest packet window without repeated sequence numbers (TCP sliding window!). Text compression: LZ77/LZ78 algorithms find non-repeating substrings as compression units. Rate limiting: longest time window without repeated request IDs.

■■ 8. Common Mistakes

- Checking if char is in map but not checking if it's within current window
- Not updating map[ch] = right after moving left (stale index remains)
- Using set.delete() to shrink window — O(n) per shrink instead of O(1) jump
- Returning 0 for empty string before checking (Go's range loop handles it)

16 Longest Repeating Character Replacement

■ 1. Problem Understanding

Given a string s and an integer k, you can replace any character in s with any other character at most k times. Return the length of the longest substring containing the same letter you can get after performing the replacement operations.

Real-World Analogy

A factory produces items labeled A or B. You can relabel at most k items. What's the longest consecutive run of same-labeled items you can achieve? Relabeling = replacing characters. Maximum consecutive run = longest window with at most k replacements needed.

Input / Output

- "ABAB", k=2 → 4 (replace both A's or both B's)
- "AABABBA", k=1 → 4 (replace one B in "AABA" or similar)

■ 2. Constraints Analysis

- 1 ≤ s.length ≤ 10^5
- s consists of uppercase English letters
- 0 ≤ k ≤ s.length
- Key formula: replacements needed in window = windowSize - maxFrequency
- Window is valid if: windowSize - maxFreq ≤ k

■ 3. Brute Force — O(n²)

Approach: For every starting index, expand and track most frequent char; check if replacements needed ≤ k.

■ 4. Optimized — Sliding Window with Max Frequency Tracking

Key Insight: Maintain sliding window. Track frequency of each character in window. Key: windowSize - maxFreq = characters to replace. If > k, shrink from left. The critical optimization: maxFreq is never decreased (only updated when we find a higher frequency), which keeps left from moving backward — O(n) total moves.

Why maxFreq doesn't decrease?

We only care about windows LONGER than the current best. Decreasing maxFreq would only happen if we need a smaller window — but we already have the best length recorded. This is a key insight that makes the algorithm O(n).

Dry Run

s="AABABBA", k=1

Window [A]: freq={A:1}, maxF=1, size=1, replace=0≤1 ✓, maxLen=1

Window [AA]: freq={A:2}, maxF=2, size=2, replace=0≤1 ✓, maxLen=2

Window [AAB]: maxF=2, size=3, replace=1≤1 ✓, maxLen=3

Window [AABA]: maxF=3, size=4, replace=1≤1 ✓, maxLen=4

Window [AABAB]: maxF=3, size=5, replace=2>1 ✗ → left++

Window [ABAB]: maxF=3(stale but ok), size=4, replace=1≤1 ✓ (not better)

Golang — Sliding Window O(n)

```
// Sliding Window — O(n) time, O(1) space (26 chars)

func characterReplacement(s string, k int) int {

count := [26]int{}

maxFreq := 0

left := 0

maxLen := 0


for right := 0; right &lt; len(s); right++ {

count[s[right]-'A']++

// Update the maximum frequency seen in current window

if count[s[right]-'A'] > maxFreq {

maxFreq = count[s[right]-'A']

}
```

```
// Characters to replace = windowSize - maxFreq

// If > k, shrink from left

for (right-left+1)-maxFreq > k {

count[s[left]-'A']--

left++

// Note: we don't update maxFreq down (optimization)

}


if right-left+1 > maxLen {

maxLen = right - left + 1

}

}

return maxLen

}
```

Time Complexity	Space Complexity
O(n)	**O(1)**

Right pointer: n steps. Left pointer: at most n steps total. Fixed-size count array of 26 = O(1) space.

5. Pattern Explanation

Pattern

Pattern: Sliding Window with character frequency tracking and 'at-most-k-replacements' validity check. The formula: invalid = windowSize - maxFreq > k. The trick of not decreasing maxFreq keeps the window expanding monotonically, giving O(n). Similar to: Max Consecutive Ones III, Minimum Size Subarray Sum.

6. Edge Cases

- k=0: no replacements → longest run of same char in original string
- k ≥ len(s): can replace all → return len(s)
- All same "AAAA" → return 4 (no replacements needed)
- Alternating "ABAB" with k=0 → return 1

7. Real-World Backend Connection

■ **Real World**
Data stream correction: in a bit stream, find the longest run of 1s achievable by flipping at most k 0s to 1s. Quality control: longest consecutive passing products if you allow k defective ones. Network packet loss tolerance: longest uninterrupted stream if k packet drops are acceptable.

■■ 8. Common Mistakes

- Recomputing maxFreq by scanning entire count array inside inner loop → O(26n) still O(n) but misses the elegant optimization
- Using while loop that decreases maxFreq — breaks the O(n) property
- Not understanding why maxFreq stays stable: we only want windows LONGER than current best

17 Longest Palindromic Substring

■ 1. Problem Understanding

Given a string s, return the longest substring that is a palindrome. A palindrome reads the same forwards and backwards.

Real-World Analogy

In genomics, palindromic DNA sequences are recognized by restriction enzymes — finding the longest such segment in a genome is a real bioinformatics problem. In text analysis, finding the longest 'mirror phrase' hidden within a larger text.

Input / Output

- "babad" → "bab" or "aba" (both valid)
- "cbbd" → "bb"
- "a" → "a"

■ 2. Constraints Analysis

- $1 \le$ s.length ≤ 1000 → $O(n^2)$ is acceptable
- s consists of digits and English letters
- Two types of palindromes: odd-length (center char) and even-length (center between chars)

■ 3. Brute Force — $O(n^3)$

Approach: Check all $O(n^2)$ substrings, verify each is palindrome in O(n). Total $O(n^3)$.

■ 4. Optimized — Expand Around Center

Key Insight: Every palindrome has a center (single character for odd-length, between two characters for even-length). Expand outward from each possible center while characters match. 2n-1 possible centers (n single chars + n-1 gaps between chars).

Golang — Expand Around Center

```
// Expand Around Center — O(n²) time, O(1) space

func longestPalindrome(s string) string {

if len(s) &lt; 2 {

return s

}

start, maxLen := 0, 1


expand := func(left, right int) {

for left >= 0 && right &lt; len(s) && s[left] == s[right] {

if right-left+1 > maxLen {

start = left

maxLen = right - left + 1

}

left--

right++

}

}


for i := 0; i &lt; len(s); i++ {

expand(i, i) // Odd-length palindromes (center = i)

expand(i, i+1) // Even-length palindromes (center between i and i+1)

}

return s[start : start+maxLen]

}
```

Time Complexity	Space Complexity
$O(n^2)$	$O(1)$

n centers, each expansion up to O(n). No auxiliary space — only two integers track the best palindrome's start and length.

■ 5. Pattern Explanation

■ Pattern

Pattern: Center Expansion for palindrome finding. Two types of centers: odd (single char) and even (between chars). Expand while matching. This is more intuitive than Manacher's algorithm (O(n)) and sufficient for n≤1000. Manacher's algorithm is the O(n) solution using the palindrome's symmetry property to avoid redundant expansion.

■ 6. Edge Cases

- Single char → return it
- All same chars 'aaaa' → whole string is palindrome
- No palindrome longer than 1: 'abcd' → return any single char
- Even palindrome: 'abba' → expand from center gap between b's

■ 7. Real-World Backend Connection

■ Real World

Bioinformatics: restriction enzyme recognition sites are palindromic DNA sequences. Data compression: palindromic patterns allow for efficient mirroring-based compression. Symmetric key generation: cryptographic systems sometimes use palindromic properties.

■■ 8. Common Mistakes

- Forgetting even-length palindromes (center between two chars)
- Returning the length instead of the actual substring
- Off-by-one: s[start:start+maxLen] not s[start:start+maxLen-1]

---

18 Palindromic Substrings (Count)

■ 1. Problem Understanding

Given a string s, return the number of palindromic substrings in it. Single characters are always palindromes.

Real-World Analogy

A linguistics professor counting ALL symmetric phrases hidden in a text, not just the longest. Every single character counts. The professor wants a total count: 'abc' has 3 palindromes (a, b, c). 'aaa' has 6 (a, a, a, aa, aa, aaa). This is a counting variant of the Longest Palindromic Substring problem.

Input / Output

- "abc" → 3 (a, b, c)
- "aaa" → 6 (a, a, a, aa, aa, aaa)

■ 2. Constraints Analysis

- 1 ≤ s.length ≤ 1000 → O(n²) acceptable
- Every single character is a palindrome → minimum answer is n
- Count each distinct (start,end) substring separately, even if same content

■ 3 & 4. Optimized — Same Center Expansion Technique

Same idea as Longest Palindromic Substring but instead of tracking max, we COUNT each valid palindrome found during expansion.

Golang — Count Palindromic Substrings

```
// Expand Around Center — O(n²) time, O(1) space

func countSubstrings(s string) int {

count := 0


expand := func(left, right int) {

for left >= 0 && right &lt; len(s) && s[left] == s[right] {

count++ // Found a palindromic substring

left--

right++

}

}


for i := 0; i &lt; len(s); i++ {

expand(i, i) // Odd-length

expand(i, i+1) // Even-length

}

return count

}
```

■ **Time Complexity** ■ **Space Complexity**

$O(n^2)$	$O(1)$

Same as Longest Palindromic Substring: 2n-1 centers, each O(n) expansion. Count incremented $O(n^2)$ times in worst case.

■ 5. Pattern Explanation

■ Pattern

Pattern: Center expansion for counting. Same as longest palindrome but we count each valid expansion instead of tracking maximum length. The counting version is often asked together with the length version to test understanding of the expand-around-center technique.

■ 6. Edge Cases

- Single char 'a' → 1
- "aa" → 3: a, a, aa
- "aaa" → 6: a,a,a,aa,aa,aaa

■ 7. Real-World Backend

■ Real World

Genome analysis: counting restriction sites. Pattern matching density analysis in text. DNA palindrome enrichment studies in computational biology.

■■ 8. Common Mistakes

- Forgetting to count single characters (they're palindromes too)
- Counting by generating substrings and checking each — $O(n^3)$
- Mixing up: this problem counts occurrences, not distinct palindromes

---

19 Minimum Window Substring

■ 1. Problem Understanding

Given strings s and t, return the minimum window substring of s such that every character in t (including duplicates) is present in the window. If no such window exists, return ".

Real-World Analogy

A Google recruiter needs to find the SHORTEST section of a candidate's portfolio that contains ALL required skills (listed in t). The portfolio is a sequence of skills demonstrated on different projects. Find the minimal contiguous block that covers all requirements.

Input / Output

- s="ADOBECODEBANC", t="ABC" → "BANC"
- s="a", t="a" → "a"
- s="a", t="aa" → "" (need 2 a's but only 1 available)

■ 2. Constraints Analysis

- 1 ≤ s.length, t.length ≤ 10^5
- Characters can be uppercase and lowercase (case-sensitive)
- Duplicates in t must be counted: t='AA' requires 2 A's in window
- Need O(n+m) solution where n=len(s), m=len(t)

■ 3. Brute Force — $O(n^2m)$

Approach: For every pair (l,r), check if window contains all of t. $O(n^2*m)$.

■ 4. Optimized — Sliding Window with Two Frequency Maps

Key Insight: Use sliding window. Expand right until window contains all t characters ('formed' condition). Then shrink left to minimize window while maintaining the condition. Track 'formed' count: number of distinct chars in t whose required frequency is met.

Step-by-step

- Build tFreq map for t
- required = number of unique chars in t
- Expand right: add s[right] to windowFreq, check if formed++
- When formed == required: try to shrink from left
- Shrink: remove s[left], if windowFreq < tFreq → formed--
- Track minimum window throughout

Golang — Sliding Window O(n+m)

```
// Sliding Window — O(n+m) time, O(n+m) space

func minWindow(s string, t string) string {

if len(t) > len(s) {

return ""

}

tFreq := make(map[byte]int)

for i := 0; i &lt; len(t); i++ {

tFreq[t[i]]++
```

```
}

required := len(tFreq) // unique chars needed
formed := 0 // unique chars with required frequency met
windowFreq := make(map[byte]int)

left := 0
minLen := len(s) + 1
minStart := 0

for right := 0; right &lt; len(s); right++ {
ch := s[right]
windowFreq[ch]++
// Check if this char's requirement is now met
if tFreq[ch] > 0 && windowFreq[ch] == tFreq[ch] {
formed++
}
// Shrink from left while window is valid
for formed == required && left &lt;= right {
if right-left+1 &lt; minLen {
minLen = right - left + 1
minStart = left
}
lch := s[left]
windowFreq[lch]--
if tFreq[lch] > 0 && windowFreq[lch] &lt; tFreq[lch] {
formed--
}
left++
}
```

```
}

if minLen > len(s) {

return ""

}

return s[minStart : minStart+minLen]

}
```

■ Time Complexity	■ Space Complexity
O(n+m)	**O(n+m)**

Right pointer moves n steps. Left pointer moves at most n steps total (only moves forward). Building tFreq: O(m). Window maps: O(n) in worst case.

■ 5. Pattern Explanation

■ Pattern

Pattern: Shrinkable Sliding Window. Expand until valid, then shrink until invalid, record minimum. The 'formed' counter elegantly tracks satisfaction of ALL character requirements in O(1) per step. This pattern (expand to satisfy + shrink to minimize) is the canonical approach for minimum window problems.

■ 6. Edge Cases

- t longer than s → impossible → return ''
- t='AA', s has only 1 A → return ''
- s == t → return s
- Case sensitivity: 'A' and 'a' are different
- t has chars not in s → return ''

■ 7. Real-World Backend Connection

■ Real World

Text search: find the shortest passage containing all query keywords. Log analysis: find the shortest time window where all required event types occurred. Gene sequencing: find shortest DNA segment containing all target motifs. Search engine snippet generation: pick the most relevant passage covering all search terms.

■■ 8. Common Mistakes

- Not handling duplicate chars in t (t='AA' needs 2 A's in window)
- Tracking total chars instead of distinct satisfied requirements ('formed')
- Not moving left all the way when window is valid (should minimize)

- Returning wrong substring bounds after loop

20 Word Break

1. Problem Understanding

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Real-World Analogy

A spell checker receives 'leetcode' (no spaces). It has a dictionary. Can this be validly segmented into known words? 'leet' + 'code' → yes! This is exactly what keyboards do with swipe text input — segment the swiped trajectory into valid words. Autocorrect needs to find valid word segmentations.

Input / Output

- s="leetcode", wordDict=["leet","code"] → true
- s="applepenapple", wordDict=["apple","pen"] → true
- s="catsandog", wordDict=["cats","dog","sand","cat","an"] → false

2. Constraints Analysis

- 1 ≤ s.length ≤ 300
- 1 ≤ wordDict.length ≤ 1000
- Words in wordDict can be reused
- Words are unique in dict

3. Brute Force — Recursion O(2^n)

Approach: Recursively try every prefix. If prefix is in dict, recurse on remainder. Exponential overlapping subproblems → needs memoization or DP.

4. Optimized — Dynamic Programming

Key Insight: dp[i] = true if s[0:i] can be segmented using dict words. For each position i, check all j < i where dp[j]=true AND s[j:i] is in dict. dp[0]=true (empty string → always valid).

Dry Run

s="leetcode", dict=["leet","code"]

dp[0]=true (base case)

i=4: j=0→dp[0]=T, s[0:4]='leet'∈dict → dp[4]=true

i=8: j=4→dp[4]=T, s[4:8]='code'∈dict → dp[8]=true

return dp[8] = true ✓

Golang — DP Solution

```
// Dynamic Programming — O(n² * m) time, O(n) space

// where n = len(s), m = avg word length

func wordBreak(s string, wordDict []string) bool {

// Build set for O(1) lookup

dictSet := make(map[string]bool, len(wordDict))

for _, w := range wordDict {

dictSet[w] = true

}


n := len(s)

dp := make([]bool, n+1)

dp[0] = true // Empty prefix is always valid


for i := 1; i &lt;= n; i++ {

for j := 0; j &lt; i; j++ {

if dp[j] && dictSet[s[j:i]] {

dp[i] = true

break // Found a valid segmentation ending at i

}

}

}

return dp[n]

}
```

Time Complexity	Space Complexity
$O(n^2 \cdot m)$	$O(n+d)$

n^2 pairs (i,j), each substring check O(m). Space: dp array O(n), dict set O(d) where d=total chars in dict.

■ 5. Pattern Explanation

■ Pattern

Pattern: 1D DP for string segmentation. dp[i] = 'can the first i characters be validly processed?' Build dp left to right, using previously computed dp[j] values. This pattern applies to: palindrome partitioning, word segmentation II (all solutions), string matching with wildcards.

■ 6. Edge Cases

- s is itself a dict word → dp[len(s)] = true
- Empty s → dp[0] = true by definition
- Dict has one-char words → can always segment if all chars are dict words
- Repeated words: 'aaaa' with dict=['a'] → dp[1]=dp[2]=dp[3]=dp[4]=true

■ 7. Real-World Backend Connection

■ Real World

Natural Language Processing: tokenization of CJK (Chinese/Japanese/Korean) text which has no spaces — same problem, just with character-based dictionary. Keyboard autocorrect and swipe input. URL parsing: segment a URL into protocol/host/path components. Compiler tokenization: segment source code into valid tokens.

■■ 8. Common Mistakes

- Not building a set from wordDict (list lookup is O(n*m) per check)
- Forgetting dp[0] = true (essential base case)
- Off-by-one: dp[i] represents s[0:i] (i-length prefix, not i-th index)
- Breaking too early in inner loop (correct: break after finding one valid j)

21 Word Search

■ 1. Problem Understanding

Given an m×n grid of characters board and a string word, return true if word exists in the grid. The word must be constructed from letters of sequentially adjacent cells (horizontally or vertically adjacent). The same cell cannot be used more than once.

Real-World Analogy

A word search puzzle from a newspaper. Given the grid and a target word, trace a path through adjacent letters that spells the word. Each cell can only be used once per path — you can't reuse a letter position. This is backtracking on a 2D grid.

Input / Output

- board=[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word="ABCCED" → true
- Same board, word="SEE" → true
- Same board, word="ABCB" → false (B reused)

■ 2. Constraints Analysis

- m, n ≥ 1; 1 ≤ word.length ≤ 15
- board and word consist of only uppercase and lowercase letters
- Adjacent means 4-directional (up/down/left/right), not diagonal
- Same cell cannot be reused → track visited cells during DFS
- Time: O(m*n*4^L) worst case where L=word length

■ 3. Approach — DFS + Backtracking

Key Insight: DFS from every cell as starting point. At each step, check if current cell matches current word character. Mark cell as visited (temporarily modify board). Recursively explore 4 neighbors for next character. Unmark cell on backtrack. Return true if all characters matched.

Backtracking Template

1. Base case: if all characters matched → return true
2. Boundary/validity check: out of bounds or cell already visited → return false
3. Character mismatch → return false
4. Mark current cell as visited (board[r][c] = '#')
5. Explore 4 directions recursively
6. Unmark cell (restore board[r][c]) — BACKTRACK

Golang — DFS + Backtracking

```
// DFS + Backtracking — O(m*n*4^L) time, O(L) space (call stack)

func exist(board [][]byte, word string) bool {

rows, cols := len(board), len(board[0])

dirs := [][2]int{{0, 1}, {0, -1}, {1, 0}, {-1, 0}}


var dfs func(r, c, idx int) bool

dfs = func(r, c, idx int) bool {

if idx == len(word) {

return true // All characters matched!
```

```
}

if r &lt; 0 || r >= rows || c &lt; 0 || c >= cols {

return false // Out of bounds

}

if board[r][c] != word[idx] {

return false // Character mismatch

}


// Mark visited by temporarily replacing with sentinel

temp := board[r][c]

board[r][c] = '#'


// Explore all 4 directions

for _, d := range dirs {

if dfs(r+d[0], c+d[1], idx+1) {

board[r][c] = temp // Restore (though not strictly needed on true)

return true

}

}


// Backtrack: restore cell

board[r][c] = temp

return false

}


for r := 0; r &lt; rows; r++ {

for c := 0; c &lt; cols; c++ {

if dfs(r, c, 0) {

return true

}
```

```
}

}

return false

}
```

■ Time Complexity	■ Space Complexity
O(m·n·4^L)	**O(L)**

*m*n starting positions. From each, DFS explores at most 4^L paths (4 directions, L depth). Space: recursion call stack depth = word length L.*

■ 5. Pattern Explanation

■ Pattern

Pattern: DFS + Backtracking on 2D grid. Template: (1) try placing at current position, (2) mark as used, (3) recurse on neighbors, (4) unmark (backtrack). This pattern applies to: N-Queens, Sudoku Solver, Word Search II (multiple words via Trie), Robot in a Grid.

■ 6. Edge Cases

- 1×1 board: word must be the single cell's character
- Word longer than total cells → impossible
- Multiple same-letter cells: DFS tries all valid starting points
- Word starts with rare character → few starting points (optimization: start from rarer end)

■ 7. Real-World Backend Connection

■ Real World

Path finding in constrained graphs: network routing where each router can only be used once. Circuit board routing: tracing wires through a grid without crossing. Game engines: pathfinding in tile-based games. Web crawlers: finding specific link sequences on a website graph without revisiting pages.

■■ 8. Common Mistakes

- Not restoring board cell on backtrack → corrupts the board for other paths
- Checking visited BEFORE boundary check → index out of bounds panic
- Using a separate visited matrix (works but wastes space; modifying board is elegant)
- Forgetting that diagonal moves are NOT allowed

■ LINKED LISTS

Problems 22–31 · Pointer Manipulation · Floyd's Algorithm · LRU Cache

22 Reverse Linked List

■ 1. Problem Understanding

Given the head of a singly linked list, reverse the list and return the reversed list's head. Each node points only to the next node; reversal means making each next pointer point backward.

Real-World Analogy

Imagine a one-way corridor where each room has a sign pointing 'next room'. Reversing means changing every sign to point back to the previous room. The first room becomes the last, and the last becomes the first. You must update each sign without losing track of where the 'old next' was.

What Problem Does This Solve?

- Browser back button: the history is stored as a linked list; navigation reverses it
- Undo operations: reversing a sequence of operations
- Palindrome checking on linked lists requires reversal
- Stack operations: LIFO structure can be implemented via reversal

Input / Output

- 1→2→3→4→5 → 5→4→3→2→1
- 1→2 → 2→1
- [] → []

■ 2. Constraints Analysis

- 0 ≤ nodes ≤ 5000
- -5000 ≤ Node.val ≤ 5000
- SINGLY linked — can only traverse forward, no back pointer
- Must return new head (formerly the last node)
- Can solve iteratively (O(1) space) or recursively (O(n) stack space)

■ 3. Approach — Iterative (Three Pointers)

Key Insight: Maintain three pointers: prev (null initially), curr (head), next (temp). At each step: save next, reverse curr's pointer to prev, advance prev and curr. After loop, prev is the new head.

Dry Run

1→2→3→null

prev=nil, curr=1: next=2, 1→nil, prev=1, curr=2

prev=1, curr=2: next=3, 2→1, prev=2, curr=3

prev=2, curr=3: next=nil,3→2, prev=3, curr=nil → stop

new head = prev = 3 → 3→2→1 ✓

Golang — Iterative and Recursive

```
type ListNode struct {
Val int
Next *ListNode
}

// Iterative — O(n) time, O(1) space
func reverseList(head *ListNode) *ListNode {
var prev *ListNode
curr := head
for curr != nil {
next := curr.Next // Save next before overwriting
curr.Next = prev // Reverse the pointer
prev = curr // Advance prev
curr = next // Advance curr
}
return prev // prev is the new head
}

// Recursive — O(n) time, O(n) space (call stack)
func reverseListRecursive(head *ListNode) *ListNode {
if head == nil || head.Next == nil {
return head // Base case: 0 or 1 node
}
```

```
newHead := reverseListRecursive(head.Next) // Reverse the rest

head.Next.Next = head // Make next node point back to current

head.Next = nil // Break original forward link

return newHead

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1) iterative / O(n) recursive**

Visit each node once. Recursive approach uses O(n) call stack depth.

■ 5. Pattern Explanation

■ Pattern

Pattern: Three-pointer technique for linked list manipulation. The prev/curr/next triplet is the fundamental tool for any in-place linked list reversal or manipulation. Always save curr.Next BEFORE changing it. This pattern appears in: reverse sublist, k-group reversal, rotate list.

■ 6. Edge Cases

- Empty list nil → return nil
- Single node → return it unchanged
- Two nodes → swap and return
- Recursive: stack overflow risk for n=5000 (but within typical Go stack limits)

■ 7. Real-World Backend Connection

■ Real World

Browser history: pressing Back reverses the navigation stack. Version control: reversing a sequence of patches. Network packet assembly: packets arriving in reverse order need reassembly. Database transaction rollback: reversing a list of applied operations.

■■ 8. Common Mistakes

- Not saving curr.Next before overwriting curr.Next → loses the rest of the list
- Not initializing prev = nil → first node's Next won't be nil in reversed list
- Returning curr instead of prev at the end (curr is nil after loop)
- Recursive: forgetting to set head.Next = nil after reversing (creates cycle)

23 Merge Two Sorted Lists

1. Problem Understanding

You are given the heads of two sorted linked lists list1 and list2. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.

Real-World Analogy

Two sorted card decks (lowest to highest). Merge them into one sorted deck without sorting again. Pick the smaller card from either deck's top. This is the classic 'merge' step in Merge Sort, and it's the key operation in merging sorted database query results.

What Problem Does This Solve?

- Merge Sort's merge step — the fundamental divide-and-conquer merge
- Database: merging sorted result sets from parallel index scans
- External sorting: merging sorted file chunks
- K-way merge: extending this to K sorted lists

Input / Output

- 1→2→4, 1→3→4 → 1→1→2→3→4→4
- [], [] → []
- [], 0 → 0

2. Constraints Analysis

- 0 ≤ nodes per list ≤ 50
- -100 ≤ Node.val ≤ 100
- Both lists are already sorted in non-decreasing order
- Must splice existing nodes — don't create new nodes with values

3. Optimized Approach — Dummy Head + Two Pointers

Key Insight: Use a dummy head node to avoid special-casing the start of the merged list. Maintain a 'tail' pointer for the merged list. At each step, compare list1 and list2 values, attach the smaller node to tail, advance both tail and that list's pointer.

Golang — Merge Two Sorted Lists

```
// Iterative — O(m+n) time, O(1) space

func mergeTwoLists(list1 *ListNode, list2 *ListNode) *ListNode {

dummy := &ListNode{} // Sentinel/dummy head
```

```
tail := dummy

for list1 != nil && list2 != nil {
if list1.Val &lt;= list2.Val {
tail.Next = list1
list1 = list1.Next
} else {
tail.Next = list2
list2 = list2.Next
}
tail = tail.Next
}
// Attach remaining nodes (one list may still have elements)
if list1 != nil {
tail.Next = list1
} else {
tail.Next = list2
}
return dummy.Next // Skip the dummy head
}

// Recursive — O(m+n) time, O(m+n) space
func mergeTwoListsRecursive(l1, l2 *ListNode) *ListNode {
if l1 == nil { return l2 }
if l2 == nil { return l1 }
if l1.Val &lt;= l2.Val {
l1.Next = mergeTwoListsRecursive(l1.Next, l2)
return l1
}
```

```
l2.Next = mergeTwoListsRecursive(l1, l2.Next)

return l2

}
```

■ Time Complexity	■ Space Complexity
O(m+n)	**O(1)**

Each node visited exactly once. Dummy head eliminates edge cases. Remaining nodes attached in O(1) via pointer.

■ 5. Pattern Explanation

■ Pattern
Pattern: Dummy Head Node + Two Pointer Merge. Dummy head simplifies code by eliminating 'if merged list is empty' edge case. This pattern is foundational for: Merge K Sorted Lists, Sort List (merge sort on linked list), intersection problems.

■ 6. Edge Cases

- Both empty → return nil
- One empty → return the other (handled by appending remaining)
- Duplicate values across lists → both included in merge
- One much longer than other → bulk attached at end

■ 7. Real-World Backend Connection

■ Real World
PostgreSQL merge join: when two tables have sorted data, the database uses this algorithm to join them. ElasticSearch: merging results from multiple sorted index segments. External sort: merging sorted file segments that don't fit in RAM. Log aggregation: merging sorted timestamped logs from multiple sources.

■■ 8. Common Mistakes

- Not attaching the remaining list after the loop (null-termination missing)
- Using >= instead of <=: breaks stability (equal elements from l1 should come first)
- Forgetting to return dummy.Next (returning dummy returns garbage sentinel)
- Modifying node values instead of relinking pointers

24 Linked List Cycle

■ 1. Problem Understanding

Given the head of a linked list, determine if the list has a cycle. A cycle exists if a node can be reached again by continuously following the next pointer. Return true if cycle exists.

Real-World Analogy

Two runners on a circular track: the fast runner will eventually lap the slow runner and they'll be at the same position. But on a straight track, the fast runner reaches the end without ever meeting the slow runner. Floyd's Cycle Detection ('tortoise and hare') is exactly this metaphor formalized as an algorithm.

What Problem Does This Solve?

- Deadlock detection: processes waiting for each other form a cycle
- Garbage collection: detecting circular references (Python GC uses cycle detection)
- Infinite loop prevention: detecting cyclic dependencies in build systems
- Currency arbitrage: detecting profit cycles in exchange rate graphs

■ 2. Constraints Analysis

- 0 ≤ nodes ≤ 10^4
- Follow-up: O(1) memory — don't use a HashSet to track visited nodes
- If no cycle, traversal terminates at null
- pos = -1 means no cycle

■ 3. Brute Force — HashSet

Approach: Traverse list, add each node's address to a set. If current node was already seen → cycle. O(n) time, O(n) space.

■ 4. Optimized — Floyd's Cycle Detection (Two Pointers)

Key Insight: Use two pointers: slow (moves 1 step), fast (moves 2 steps). If no cycle, fast reaches null. If cycle, fast eventually 'laps' slow and they meet at the same node. This uses O(1) space — no extra data structures.

Why They Must Meet

Once both pointers are in the cycle, the distance between them decreases by 1 each step (fast gains 1 on slow per iteration). They will meet within 'cycle_length' steps.

Golang — Floyd's Tortoise and Hare

```
// Floyd's Cycle Detection — O(n) time, O(1) space

func hasCycle(head *ListNode) bool {

slow, fast := head, head

for fast != nil && fast.Next != nil {
```

```
slow = slow.Next // Move 1 step

fast = fast.Next.Next // Move 2 steps

if slow == fast { // Pointer equality (same node, not same value!)

return true

}

}

return false // fast reached end → no cycle

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Without cycle: fast reaches end in n/2 steps. With cycle: pointers meet within n steps. O(1) space — only two pointers.

■ 5. Pattern Explanation

■ Pattern

Pattern: Floyd's Cycle Detection. Fast/slow pointers — if a cycle exists, the fast pointer must eventually catch the slow pointer. If no cycle, fast pointer exits. Extension: Linked List Cycle II (find entry point), Find Duplicate Number (treat array as linked list), Happy Number.

■ 6. Edge Cases

- Empty list nil → no cycle (fast starts nil → loop doesn't execute)
- Single node pointing to itself → cycle (fast=slow after 1 step)
- Single node with no next → no cycle
- Very long list with cycle at end → O(n) steps to detect

■ 7. Real-World Backend Connection

■ Real World

Python's garbage collector uses cycle detection to handle circular references (objects pointing to each other). Linux kernel's wait_for_completion detects deadlock cycles. Package managers (npm, pip) detect circular dependencies using cycle detection. Database foreign key constraint checking.

■■ 8. Common Mistakes

- Comparing node VALUES instead of node POINTERS — different nodes can have same value
- Not checking fast.Next != nil before fast.Next.Next (nil pointer dereference)

• Initializing both to head → they start equal → false positive if you check before moving

25 Remove Nth Node From End of List

■ 1. Problem Understanding

Given the head of a linked list, remove the nth node from the end of the list and return its head. Solve it in one pass.

Real-World Analogy

A parade of floats passes by. You want to remove the 3rd-from-last float without knowing the total count in advance. Solution: deploy a 'scout' that's always 3 floats ahead. When the scout reaches the end, your main position is exactly 3 from the end — remove that float.

Input / Output

- 1→2→3→4→5, n=2 → 1→2→3→5 (removed 4, which is 2nd from end)
- 1, n=1 → [] (only node)
- 1→2, n=1 → 1 (removed last)

■ 2. Constraints Analysis

- 1 ≤ nodes ≤ 30, 1 ≤ n ≤ nodes
- n is always valid (guaranteed)
- Removing first node: special case — return head.Next
- One pass preferred — use two pointers with n-step offset

■ 3. Optimized — Two Pointers with Gap of N

Key Insight: Create two pointers with a gap of n+1 between them. Use a dummy head to simplify edge cases. Advance fast pointer n+1 steps. Then advance both until fast reaches null. Slow is then AT the node BEFORE the one to delete.

Golang — Two Pointer One Pass

```
// Two Pointers One Pass — O(n) time, O(1) space

func removeNthFromEnd(head *ListNode, n int) *ListNode {

dummy := &ListNode{Next: head}

slow, fast := dummy, dummy


// Advance fast n+1 steps ahead
```

```
for i := 0; i &lt;= n; i++ {
fast = fast.Next
}

// Move both until fast reaches end
for fast != nil {
slow = slow.Next
fast = fast.Next
}

// slow.Next is the node to remove
slow.Next = slow.Next.Next
return dummy.Next
}
```

■ Time Complexity	■ Space Complexity
O(L)	**O(1)**

L = list length. Single pass. Dummy head handles edge cases (removing head). Two pointers, no extra space.

■ 5. Pattern Explanation

■ Pattern

Pattern: Two Pointers with fixed gap for 'from the end' problems. Setting a gap of n between two pointers lets you simultaneously track a position that's n steps behind the front pointer. Dummy head eliminates 'is it the first node?' edge case. Applies to: find middle of list (gap=n/2), split list at k-th from end.

■ 6. Edge Cases

- Remove the ONLY node (n=1, list has 1 element) → dummy handles it
- Remove the HEAD (n = list length) → dummy.Next = head.Next
- Remove the TAIL (n=1) → slow points to second-to-last, slow.Next = nil

■ 7. Real-World Backend Connection

■ **Real World**
Network protocol: remove the Nth packet from the end of a transmission buffer without knowing total size. Log rotation: remove the oldest N log entries. Undo history management: keep only the last K operations by removing from the end.

■■ 8. Common Mistakes

- Advancing fast by n steps instead of n+1 (slow ends up ON node to remove, not BEFORE it)
- Not using dummy head → special case when removing head node
- Off-by-one in gap setup → removes wrong node

26 Reorder List

■ 1. Problem Understanding

Given a singly linked list L: L0→L1→…→Ln-1→Ln, reorder it to: L0→Ln→L1→Ln-1→L2→Ln-2→… You may not modify node values, only node pointers.

Real-World Analogy

A book club reads from both ends of their reading list simultaneously — the first pick from the front, then the first from the back, alternating. The reordered list represents this alternating selection pattern. Weaving two sequences together.

Input / Output

- 1→2→3→4 → 1→4→2→3
- 1→2→3→4→5 → 1→5→2→4→3

■ 2. Approach — Three Steps

Key Insight: Decompose into three classic operations: (1) Find middle of list using fast/slow pointers. (2) Reverse the second half. (3) Merge the two halves alternately.

Golang — Find Middle + Reverse + Merge

```
// Three-Step: Find Middle + Reverse + Merge — O(n) time, O(1) space

func reorderList(head *ListNode) {

if head == nil || head.Next == nil {

return

}
```

```
// Step 1: Find middle using slow/fast pointers

slow, fast := head, head

for fast.Next != nil && fast.Next.Next != nil {

slow = slow.Next

fast = fast.Next.Next

}

// slow is now at the middle


// Step 2: Reverse second half (slow.Next to end)

second := slow.Next

slow.Next = nil // Split the list

var prev *ListNode

for second != nil {

next := second.Next

second.Next = prev

prev = second

second = next

}

second = prev // second now points to reversed second half


// Step 3: Merge two halves alternately

first := head

for second != nil {

tmp1 := first.Next

tmp2 := second.Next

first.Next = second

second.Next = tmp1

first = tmp1

second = tmp2
```

```
}

}
```

■ Time Complexity	■ Space Complexity
O(n)	O(1)

Three linear passes. No extra data structures (arrays or stacks would be O(n) space).

■ 5. Pattern Explanation

■ Pattern

Pattern: Composite linked list technique. Complex linked list transformations often decompose into: finding middle + reversing + merging. This three-step pattern appears in: Palindrome Linked List, Merge Sort on Linked List, Interleaving two lists.

■ 6. Edge Cases

- 1 node → no change
- 2 nodes → [1,2] → [1,2] (unchanged — L0→L1 is already correct)
- Even vs odd length: split correctly at middle

■ 7. Real-World Backend

■ Real World

Load balancing: interleaving requests from two priority queues. Playlist generation: alternating between two sorted playlists. Data interleaving in storage systems: RAID striping interleaves data across drives.

■■ 8. Common Mistakes

- Not splitting the list (slow.Next=nil) before reversing → reversal corrupts first half
- Wrong stopping condition: fast.Next!=nil && fast.Next.Next!=nil (not fast!=nil)
- Losing track of next pointers during merge (must save both tmp1 and tmp2)

27 Add Two Numbers

■ 1. Problem Understanding

Two non-empty linked lists represent two non-negative integers. Digits are stored in REVERSE order (least significant digit first). Each node contains a single digit. Add the two numbers and return the sum as a linked list

in the same reverse format.

Real-World Analogy

Big integer arithmetic: numbers too large for CPU registers (thousands of digits). Store them as linked lists, add digit by digit from least significant to most significant, handling carries. This is exactly how Python's arbitrary-precision integers work internally.

Input / Output

- l1=2→4→3, l2=5→6→4 → 7→0→8 (342 + 465 = 807)
- l1=0, l2=0 → 0
- l1=9→9→9→9→9→9→9, l2=9→9→9→9 → 8→9→9→9→0→0→0→1 (9999999+9999=10009998)

■ 2. Constraints Analysis

- 1 ≤ nodes ≤ 100
- 0 ≤ Node.val ≤ 9
- Lists don't have leading zeros (except the number 0 itself)
- Digits in REVERSE order — this is convenient! We add from least significant.
- Handle carry propagation, including final carry beyond both lists

■ 3. Approach — Digit-by-Digit with Carry

Key Insight: Simulate grade-school addition. Process nodes simultaneously. At each step: sum = l1.val + l2.val + carry. Result digit = sum % 10. New carry = sum / 10. Continue until BOTH lists exhausted AND carry = 0.

Golang — Grade-School Addition

```go
// Simulate Addition — O(max(m,n)) time, O(max(m,n)) space

func addTwoNumbers(l1 *ListNode, l2 *ListNode) *ListNode {

dummy := &ListNode{}

curr := dummy

carry := 0


for l1 != nil || l2 != nil || carry > 0 {

sum := carry

if l1 != nil {

sum += l1.Val

l1 = l1.Next

}
```

```
if l2 != nil {
sum += l2.Val
l2 = l2.Next
}
carry = sum / 10
curr.Next = &ListNode{Val: sum % 10}
curr = curr.Next
}
return dummy.Next
}
```

■ Time Complexity	■ Space Complexity
O(max(m,n))	**O(max(m,n))**

Process each node once. Output length = max(m,n) + 1 (possible carry). Space for output list.

■ 5. Pattern Explanation

■ Pattern

Pattern: Parallel list traversal with carry propagation. Process two lists simultaneously, handle unequal lengths, propagate remainder (carry). Same pattern: Subtract Two Numbers, Multiply Big Integers. The 'continue while carry > 0' is the critical termination condition — don't forget the final carry digit!

■ 6. Edge Cases

- Different lengths: l1 may be longer → handle nil gracefully
- Final carry: 9→9 + 9→9 = 8→9→1, length n+1
- Both zero: 0 + 0 = 0
- Sum = 10: single node 1→0 → creates two-node result 0→1

■ 7. Real-World Backend Connection

■ Real World

Arbitrary precision arithmetic in Python, Java BigInteger. Cryptographic systems: RSA operates on numbers with hundreds of digits. Financial systems: currency calculations requiring exact precision use big decimal arithmetic with the same carry-propagation logic.

■■ 8. Common Mistakes

- Forgetting final carry after both lists exhausted → missing most significant digit
- Not handling different-length lists → null pointer on shorter list
- Storing multi-digit sum in node (each node holds ONE digit)

28 Linked List Cycle II — Find Entry Point

■ 1. Problem Understanding

Given head of linked list with possible cycle, return the NODE where the cycle begins. If no cycle, return nil.

Real-World Analogy

Two runners on a track with a loop: the track goes straight for a while, then enters a circular section. When the fast runner laps the slow runner inside the loop, a mathematical relationship tells us exactly where the loop starts. This is the 'entry point' detection.

Mathematical Proof

Let: F = distance from head to cycle start, C = cycle length, a = distance from cycle start to meeting point.

When slow and fast meet: slow traveled F+a steps, fast traveled F+a+nC steps.

Since fast moves 2x: 2(F+a) = F+a+nC → F = nC - a.

This means: from meeting point, a+F = nC = full cycle(s). So a pointer from HEAD and a pointer from MEETING POINT, both moving 1 step at a time, meet at CYCLE ENTRY!

Golang — Floyd's Cycle Entry Detection

```
// Floyd's Cycle Entry Detection — O(n) time, O(1) space

func detectCycle(head *ListNode) *ListNode {

slow, fast := head, head


// Phase 1: Detect cycle

for fast != nil && fast.Next != nil {

slow = slow.Next

fast = fast.Next.Next

if slow == fast {

// Phase 2: Find entry point
```

```
// Move one pointer to head, keep other at meeting point

// Both move at same speed until they meet → that's the cycle entry

entry := head

for entry != slow {

entry = entry.Next

slow = slow.Next

}

return entry

}

}

return nil // No cycle

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Phase 1: detect cycle in O(n). Phase 2: find entry in O(F) where F ≤ n. Total O(n). Only two pointers — O(1) space.

■ 5. Pattern Explanation

■ Pattern

Pattern: Floyd's Algorithm extended. Phase 1 detects a cycle. Phase 2 uses the mathematical relationship F = nC - a to find the entry point. This is a non-obvious but elegant application of cycle detection mathematics. Same pattern used in: Find Duplicate Number (array as implicit linked list).

■ 6. Edge Cases

- No cycle → return nil
- Cycle at head (pos=0): meeting point analysis still works
- Single node with self-loop → fast=slow=head from start, detected immediately

■ 7. Real-World Backend

■ **Real World**
Circular dependency detection in package managers: finding WHICH package introduces the cycle. Deadlock analysis: identifying the starting process in a wait-for cycle. Database transaction deadlock: finding the first transaction in the deadlock ring.

■■ 8. Common Mistakes

- Just returning true/false instead of the actual cycle entry node
- Moving fast pointer to head instead of slow (mathematical proof requires specific pointer)
- Not understanding WHY the proof works — just implementing the code pattern

29 Find the Duplicate Number

■ 1. Problem Understanding

Given an array nums containing n+1 integers where each integer is in [1,n], there is exactly one duplicate. Return the duplicate without modifying the array and using O(1) extra space.

Real-World Analogy

A hotel has n rooms numbered 1 to n. n+1 guests check in, each given a room number. One room number was assigned twice. The guest registry can't be modified, and you can only use a notepad. Which room was double-booked?

Key Insight — Treat as Cycle Detection!

Think of nums as a function f(i) = nums[i]. The sequence i → nums[i] → nums[nums[i]] → ... forms a 'functional graph.' Since one value repeats, there's a cycle. The entry point of the cycle IS the duplicate! Apply Floyd's cycle detection on this implicit linked list.

Input / Output

- nums=[1,3,4,2,2] → 2
- nums=[3,1,3,4,2] → 3

■ 2. Constraints Analysis

- Cannot modify array (no sorting or marking)
- O(1) extra space (no HashSet)
- Exactly one duplicate value, but may appear more than twice
- Range [1,n] ensures all values are valid 'next' indices

■ 3. Optimized — Floyd's Cycle Detection on Array

Golang — Floyd's on Array (Cycle Entry = Duplicate)

```
// Floyd's Cycle Detection — O(n) time, O(1) space

func findDuplicate(nums []int) int {

// Phase 1: Find intersection point

slow := nums[0]

fast := nums[0]

for {

slow = nums[slow] // Move 1 step

fast = nums[nums[fast]] // Move 2 steps

if slow == fast {

break

}

}


// Phase 2: Find cycle entry = duplicate number

entry := nums[0]

for entry != slow {

entry = nums[entry]

slow = nums[slow]

}

return entry

}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(1)**

Same as Floyd's cycle detection. Treating array as implicit linked list. No extra space — only two index pointers.

■ 5. Pattern Explanation

■ **Pattern**
Pattern: Reduce to known problem (Cycle Detection). The key insight is modeling the array as an implicit linked list where nums[i] is the 'next pointer' of node i. The duplicate value creates two entries pointing to the same 'node', which manifests as a cycle. The cycle entry = the duplicate.

■ 6. Edge Cases

- Duplicate appears more than twice → still finds it (cycle entry is unambiguous)
- Duplicate is 1 → cycle starts at index 1
- nums=[1,1]: both 1's point to index 1 → cycle of length 1 at index 1

■ 7. Real-World Backend Connection

■ **Real World**
Hash collision detection: finding which key hash collides (implicit graph structure). ID deduplication in constrained memory systems. Database: finding the first duplicate primary key in a bounded integer sequence without extra storage.

■■ 8. Common Mistakes

- Starting both slow and fast at index 0 (not nums[0]) — must use value as next pointer
- Using binary search approach: O(n log n) — valid but not as elegant
- Assuming duplicate appears exactly twice — may appear more often

---

30 Copy List With Random Pointer

■ 1. Problem Understanding

A linked list where each node has a 'random' pointer that points to any node in the list or null. Construct a deep copy of this list. The deep copy must have completely NEW nodes with the same val, next, and random pointer structure, but no original node pointers.

Real-World Analogy

A contact list where each person has a 'reference' to any other person (e.g., their emergency contact). Deep copying this means creating entirely new contact cards with all the same information AND updating the emergency contact references to point to the NEW cards, not the originals.

■ 2. Constraints Analysis

- $0 \le n \le 1000$
- Random pointer may point to any node, or nil

- Must create NEW nodes — cannot reference original nodes
- Deep copy: original.random → new.random must be the COPY of the pointed node

■ 3. Optimized — HashMap (Original → Copy)

Key Insight: Two-pass approach. Pass 1: create all new nodes, build map[original]→copy. Pass 2: set next and random pointers using the map to look up the copies.

Golang — HashMap Deep Copy

```
type Node struct {
Val int
Next *Node
Random *Node
}

// HashMap approach — O(n) time, O(n) space
func copyRandomList(head *Node) *Node {
if head == nil {
return nil
}

// Map from original node to its copy
nodeMap := make(map[*Node]*Node)

// Pass 1: Create all new nodes
curr := head
for curr != nil {
nodeMap[curr] = &Node{Val: curr.Val}
curr = curr.Next
}

// Pass 2: Set Next and Random pointers
curr = head
for curr != nil {
```

```
if curr.Next != nil {
nodeMap[curr].Next = nodeMap[curr.Next]
}
if curr.Random != nil {
nodeMap[curr].Random = nodeMap[curr.Random]
}
curr = curr.Next
}

return nodeMap[head]
}
```

■ Time Complexity	■ Space Complexity
O(n)	**O(n)**

Two passes through n nodes. HashMap stores n entries. Space O(n) for the map plus O(n) for new nodes.

■ 5. Pattern Explanation

■ Pattern

Pattern: HashMap for node mapping in graph/list copy problems. Whenever deep copying a structure with arbitrary cross-references, first create all new nodes (Pass 1), then wire up all pointers using the original→copy map (Pass 2). Also works for: Clone Graph, Clone N-ary Tree.

■ 6. Edge Cases

- Empty list → return nil
- Random pointing to itself → nodeMap[curr] = nodeMap[curr.Random] ← same new node
- Random pointing to nil → set copy.Random = nil
- All randoms pointing to head → copy randoms point to copy head

■ 7. Real-World Backend Connection

■ Real World

Deep copy of object graphs in serialization: Protocol Buffers deep copy. Game state cloning: saving game state requires deep copying all interconnected objects. Distributed systems: replicating data structures with internal cross-references across nodes. Compiler: copying AST nodes with symbol table references.

■■ 8. Common Mistakes

- Shallow copy: copying node objects but keeping original random pointers
- Setting pointers in same pass as creation — referenced copies may not exist yet
- Not handling nil random pointer → nil pointer dereference on nodeMap lookup

---

31 LRU Cache

■ 1. Problem Understanding

Design a data structure that follows the Least Recently Used (LRU) cache eviction policy. Implement LRUCache with capacity, get(key) → returns value or -1, and put(key, value) → inserts/updates, evicting LRU item if at capacity. Both operations must be O(1).

Real-World Analogy

A desk with limited space. You keep your most recently used books on the desk. When the desk is full and you need a new book, you remove the book you used LEAST recently (it's gathering dust). LRU cache models exactly this behavior for computer memory: keep hot data, evict cold data.

What Problem Does This Solve?

- CPU cache management: hardware L1/L2/L3 caches use LRU variants
- OS page replacement: operating system page tables use LRU
- Database buffer pool: keep hot database pages in memory
- Web/API caching: CDN cache eviction policies

■ 2. Constraints Analysis

- 1 ≤ capacity ≤ 3000
- O(1) for BOTH get and put — this rules out simple arrays or sorted structures
- O(1) requirement means: O(1) lookup AND O(1) move-to-front AND O(1) evict-from-back
- Solution: HashMap (O(1) lookup) + Doubly Linked List (O(1) insert/delete)

■ 3. Data Structure Design

Key Insight: Combine two data structures: HashMap (key → node pointer) for O(1) lookup, and Doubly Linked List for O(1) ordering (most recent at front, least recent at back). Sentinel head and tail nodes eliminate edge cases.

On GET: look up node in map, move it to front (most recently used)

On PUT: if exists, update and move to front. If new: add to front, if over capacity, evict from back.

```
Golang — LRU Cache (HashMap + Doubly Linked List)
```

```
// LRU Cache — O(1) get and put

type LRUNode struct {

key, val int

prev, next *LRUNode

}


type LRUCache struct {

capacity int

cache map[int]*LRUNode

head *LRUNode // Most recently used sentinel

tail *LRUNode // Least recently used sentinel

}


func Constructor(capacity int) LRUCache {

head := &LRUNode{}

tail := &LRUNode{}

head.next = tail

tail.prev = head

return LRUCache{

capacity: capacity,

cache: make(map[int]*LRUNode),

head: head,

tail: tail,

}

}


func (c *LRUCache) remove(node *LRUNode) {

node.prev.next = node.next

node.next.prev = node.prev

}
```

```
func (c *LRUCache) insertFront(node *LRUNode) {

node.next = c.head.next

node.prev = c.head

c.head.next.prev = node

c.head.next = node

}


func (c *LRUCache) Get(key int) int {

if node, ok := c.cache[key]; ok {

c.remove(node)

c.insertFront(node) // Mark as most recently used

return node.val

}

return -1

}


func (c *LRUCache) Put(key int, value int) {

if node, ok := c.cache[key]; ok {

c.remove(node)

node.val = value

c.insertFront(node)

} else {

node := &LRUNode{key: key, val: value}

c.cache[key] = node

c.insertFront(node)

if len(c.cache) > c.capacity {

// Evict LRU: the node just before tail

lru := c.tail.prev

c.remove(lru)
```

```
delete(c.cache, lru.key)

}

}

}
```

■ Time Complexity	■ Space Complexity
O(1) per operation	**O(capacity)**

HashMap: O(1) lookup. Doubly linked list: O(1) insert/remove with direct pointer. Sentinel head/tail eliminate nil checks. Space: O(capacity) for both structures.

■ 5. Pattern Explanation

■ Pattern

Pattern: HashMap + Doubly Linked List for O(1) ordered operations. HashMap gives O(1) access to ANY node. Doubly Linked List gives O(1) reordering and eviction because each node has prev/next pointers. The combination powers LRU, LFU, and other ordered cache designs. This design is used directly in production Redis, Memcached, and OS kernels.

■ 6. Edge Cases

- Get key that doesn't exist → return -1
- Put to update existing key → move to front, update value
- Capacity = 1: every put evicts the previous entry
- Same key put twice: second put should update, not add

■ 7. Real-World Backend Connection

■ Real World

Redis's MAXMEMORY policy uses LRU eviction. Linux kernel page cache (pagecache) uses a variant of LRU. Chrome browser maintains visited page cache with LRU. CPU hardware cache controllers implement LRU in silicon. Database buffer pools (PostgreSQL's shared_buffers) use Clock algorithm — a hardware-friendly LRU approximation.

■■ 8. Common Mistakes

- Not storing the KEY in the DLL node — needed when evicting to delete from map
- Singly linked list: can't remove node in O(1) without traversal
- Using OrderedDict/list instead of custom DLL: may not satisfy O(1) in all operations
- Not handling capacity overflow check after insert (off-by-one: > vs >=)
- Missing sentinel head/tail → complex edge cases for empty list or single element

■ STACK

Problems 32–38 · Monotonic Stack · Backtracking · RPN Evaluation

Problem 32 Valid Parentheses

■ Real-World Analogy & What Problem It Solves

Imagine reading source code. Every opening brace '{' must be closed by its matching '}' in the correct order. Compilers, IDEs, and linters use this exact algorithm to validate syntax. Without it, you'd have no bracket-mismatch errors, and broken code would silently compile.

> **■ Real World**
> Real-World Use: XML/HTML parsers validate tag nesting, compilers validate expression grouping, calculators validate formula syntax — all using stack-based matching.

■ Problem Statement

Given a string s containing only '(', ')', '{', '}', '[', ']', determine if the input string is valid. Open brackets must be closed by the same type of brackets, in the correct order.

■ Constraints Analysis

- 1 ≤ s.length ≤ 10■ — can be long; O(n) needed
- Only bracket characters — no need to skip non-brackets
- Stack depth ≤ n/2 — memory bounded by input size

■ Intuition & Pattern

The key insight: when you see a closing bracket, it MUST match the most recently opened bracket. This 'most recently opened' constraint is exactly what a stack gives you — LIFO (Last In, First Out). Push opening brackets. When you see a closing bracket, check if the top of the stack is its matching opener.

■■ Approach: Stack + HashMap

1. Build a map: closing → opening (')':'(', '}':'{', ']':'[') 2. Iterate each character: - If opening bracket → push onto stack - If closing bracket → pop from stack; if stack empty or popped ≠ expected opener → return false 3. After full scan → return stack.IsEmpty() (no unmatched openers left)

■■ Golang Implementation

```
func isValid(s string) bool {

stack := []rune{}

match := map[rune]rune{

')': '(',
```

```
'}': '{',
']': '[',
}
for _, ch := range s {
if ch == '(' || ch == '{' || ch == '[' {
stack = append(stack, ch)
} else {
if len(stack) == 0 || stack[len(stack)-1] != match[ch] {
return false
}
stack = stack[:len(stack)-1] // pop
}
}
return len(stack) == 0
}
```

■ Dry Run: s = "{[]}"

ch='{' → stack:[{] ch='[' → stack:[{, []] ch=']' → match[']']='[', top='[' ✓ → pop → stack:[{] ch='}' → match['}']='{', top='{' ✓ → pop → stack:[] stack empty → return true ✓

■ Time Complexity	■ Space Complexity
O(n)	O(n)

■ Insight

Edge cases: empty string → true (stack stays empty). Odd length → automatically false (unmatched bracket). Only closers ')]]' → stack empty on first pop → false.

Problem 33 Min Stack

■ Real-World Analogy & What Problem It Solves

Think of a trading system tracking 'running minimum price seen so far.' Each new price pushed onto the history must also record the minimum at that point in time. When you undo (pop) a trade, you must still know the

minimum before that trade. Without this pattern, you'd need a full O(n) scan every time someone asks 'what was the cheapest price ever?'

■ Problem Statement

Design a stack that supports push, pop, top, and retrieving the minimum element — all in O(1) time.

■ Intuition & Pattern: Auxiliary Min-Stack

Maintain TWO stacks: the main stack and a 'min stack'. The min stack mirrors the main stack but each position stores 'the minimum of all elements from bottom up to this position.' When you push x, push min(x, minStack.top) onto the min stack. When you pop, pop both stacks. GetMin() simply returns minStack.top — always O(1).

■■ Golang Implementation

```
type MinStack struct {
stack []int
minStack []int
}

func Constructor() MinStack {
return MinStack{}
}

func (s *MinStack) Push(val int) {
s.stack = append(s.stack, val)
minVal := val
if len(s.minStack) > 0 && s.minStack[len(s.minStack)-1] &lt; val {
minVal = s.minStack[len(s.minStack)-1]
}
s.minStack = append(s.minStack, minVal)
}

func (s *MinStack) Pop() {
s.stack = s.stack[:len(s.stack)-1]
s.minStack = s.minStack[:len(s.minStack)-1]
}
```

```
func (s *MinStack) Top() int {
return s.stack[len(s.stack)-1]
}

func (s *MinStack) GetMin() int {
return s.minStack[len(s.minStack)-1]
}
```

■ Dry Run: Push 5,3,7,2 → GetMin → Pop → GetMin

Push 5: stack=[5], minStack=[5] Push 3: stack=[5,3], minStack=[5,3] Push 7: stack=[5,3,7], minStack=[5,3,3] Push 2: stack=[5,3,7,2], minStack=[5,3,3,2] GetMin → minStack.top = 2 ✓ Pop: stack=[5,3,7], minStack=[5,3,3] GetMin → minStack.top = 3 ✓ (correctly restored!)

■ Time Complexity	■ Space Complexity
O(1) all ops	O(n)

Problem 34 Evaluate Reverse Polish Notation

■ Real-World Analogy & What Problem It Solves

RPN (Postfix notation) is how calculators and compilers actually evaluate expressions internally. When your CPU evaluates '3 + 4 * 2', it converts it to postfix '3 4 2 * +' using a stack. RPN eliminates the need for parentheses and operator precedence rules entirely — making it perfect for stack-based virtual machines like the JVM and CPython.

■ Problem Statement

Evaluate the value of an arithmetic expression in Reverse Polish Notation. Valid operators: +, -, *, /. Each operand may be an integer or another expression. Division truncates toward zero.

■ Intuition & Pattern

Rule: Numbers → push. Operator → pop TWO numbers, apply operator, push result. The stack always holds the 'pending operands.' Final answer is the last item in the stack.

■■ Golang Implementation

```
func evalRPN(tokens []string) int {
stack := []int{}
for _, tok := range tokens {
switch tok {
```

```
case "+", "-", "*", "/":
b := stack[len(stack)-1]
a := stack[len(stack)-2]
stack = stack[:len(stack)-2]
switch tok {
case "+": stack = append(stack, a+b)
case "-": stack = append(stack, a-b)
case "*": stack = append(stack, a*b)
case "/": stack = append(stack, a/b) // Go truncates toward zero
}
default:
num, _ := strconv.Atoi(tok)
stack = append(stack, num)
}
}
return stack[0]
}
```

■ Dry Run: ["2","1","+","3","*"]

tok="2" → push → stack:[2] tok="1" → push → stack:[2,1] tok="+" → pop 1,2 → 2+1=3 → stack:[3] tok="3" → push → stack:[3,3] tok="*" → pop 3,3 → 3*3=9 → stack:[9] Result: 9 ✓

■ Time Complexity	■ Space Complexity
O(n)	O(n)

Problem 35 Generate Parentheses

■ Real-World Analogy & What Problem It Solves

Generating all valid bracket combinations is the foundation of generating all valid syntax trees, expression trees, and recursive grammar structures. Template engines, code generators, and combinatorial test-case generators use this exact pattern to enumerate all valid structures for a given grammar rule.

■ Problem Statement

Given n pairs of parentheses, generate all combinations of well-formed parentheses.

■ Intuition & Pattern: Backtracking with Constraints

At each step, you can add '(' if open count < n, or ')' if close count < open count. These two constraints ensure we never exceed n pairs and never close before opening. This prunes the recursion tree from 2^(2n) to the Catalan number C(n) — drastically smaller.

■■ Golang Implementation

```
func generateParenthesis(n int) []string {

result := []string{}

var backtrack func(cur string, open, close int)

backtrack = func(cur string, open, close int) {

if len(cur) == 2*n {

result = append(result, cur)

return

}

if open &lt; n {

backtrack(cur+"(", open+1, close)

}

if close &lt; open {

backtrack(cur+")", open, close+1)

}

}

backtrack("", 0, 0)

return result

}
```

■ Dry Run: n=2

"" open=0 close=0 → "(" open=1 → "((" open=2 → "(())" close=1 → "(())" ✓ DONE → "()" close=1 → "()(" open=2 → "()()" ✓ DONE Result: ["(())", "()()"] ✓

■ Time Complexity	■ Space Complexity
O(4■/√n) Catalan	O(n) recursion depth

Problem 36 Daily Temperatures

■ Real-World Analogy & What Problem It Solves

This is the 'next greater element' problem — one of the most common in systems programming. Stock market systems use it to find 'how many days until a stock price rises above today.' Event-driven systems use it to find 'next event of higher priority.' CPU scheduling uses it for 'next task with higher priority in the queue.'

> **■ Real World**
> Real-World Use: Histogram analysis, stock trading signals, queue scheduling, and any system that needs 'next occurrence of a larger/smaller value.'

■ Problem Statement

Given an array of daily temperatures, return an array answer such that answer[i] is the number of days until a warmer temperature. If no future warmer day exists, answer[i]=0.

■ Intuition & Pattern: Monotonic Stack

A monotonic stack maintains elements in sorted (monotonically decreasing here) order. We push indices onto the stack. When we encounter a temperature higher than the stack's top, we've found the 'next warmer day' for all stack elements that it beats. Pop them and record the gap. This ensures each element is pushed and popped at most once → O(n) total.

■■ Golang Implementation

```
func dailyTemperatures(temperatures []int) []int {

n := len(temperatures)

answer := make([]int, n)

stack := []int{} // stores indices


for i, temp := range temperatures {

// While current temp is warmer than temp at stack top

for len(stack) > 0 && temperatures[stack[len(stack)-1]] &lt; temp {

idx := stack[len(stack)-1]

stack = stack[:len(stack)-1]

answer[idx] = i - idx // days waited

}

stack = append(stack, i)
```

```
}

return answer

// Remaining indices in stack have answer=0 (no warmer day)

}
```

■ Dry Run: [73,74,75,71,69,72,76,73]

i=0,T=73: stack:[0] i=1,T=74: 74>73 → pop 0, ans[0]=1-0=1. stack:[1] i=2,T=75: 75>74 → pop 1, ans[1]=2-1=1. stack:[2] i=3,T=71: 71<75 → push. stack:[2,3] i=4,T=69: 69<71 → push. stack:[2,3,4] i=5,T=72: 72>69 → pop 4,ans[4]=1. 72>71 → pop 3,ans[3]=2. 72<75 → push. stack:[2,5] i=6,T=76: 76>72 → pop 5,ans[5]=1. 76>75 → pop 2,ans[2]=4. stack:[6] i=7,T=73: 73<76 → push. stack:[6,7] End: indices 6,7 → ans[6]=ans[7]=0 Result: [1,1,4,2,1,1,0,0] ✓

■ Time Complexity	■ Space Complexity
O(n)	O(n)

Problem 37 Car Fleet

■ Real-World Analogy & What Problem It Solves

This models highway traffic simulation — a critical domain in urban planning and autonomous driving. Cars traveling toward a destination form 'fleets' when faster cars catch up to slower ones ahead. Traffic simulators, GPS routing, and autonomous vehicle systems compute fleet formation to predict congestion and optimize routing.

■ Problem Statement

n cars travel toward a target. position[i] and speed[i] describe each car. A car cannot pass another; it slows down to form a fleet. Return the number of car fleets that reach the target.

■ Intuition & Pattern

Sort cars by starting position (rightmost first → closest to target). Compute time_to_target = (target - pos) / speed for each. If a car behind takes ≤ time of the car ahead, it catches up and joins that fleet (same arrival time). Use a stack: push arrival times. If new car's time ≤ stack top, it merges (don't push). Otherwise it forms a new fleet (push).

■■ Golang Implementation

```
func carFleet(target int, position []int, speed []int) int {

n := len(position)

type Car struct{ pos, spd int }
```

```
cars := make([]Car, n)

for i := range cars {

cars[i] = Car{position[i], speed[i]}

}

// Sort by position descending (closest to target first)

sort.Slice(cars, func(i, j int) bool {

return cars[i].pos > cars[j].pos

})


stack := []float64{} // arrival times

for _, car := range cars {

time := float64(target-car.pos) / float64(car.spd)

// If this car arrives after or at same time as fleet ahead: new fleet

if len(stack) == 0 || time > stack[len(stack)-1] {

stack = append(stack, time)

}

// else: merges with fleet ahead (don't push)

}

return len(stack)

}
```

■ Dry Run: target=12, pos=[10,8,0,5,3], spd=[2,4,1,1,3]

Sorted by pos desc: [(10,2),(8,4),(5,1),(3,3),(0,1)] Times: (12-10)/2=1.0, (12-8)/4=1.0, (12-5)/1=7.0, (12-3)/3=3.0, (12-0)/1=12.0 stack=[]: push 1.0 → [1.0] time=1.0: 1.0 ≤ 1.0 → merge (don't push) → [1.0] time=7.0: 7.0 > 1.0 → push → [1.0, 7.0] time=3.0: 3.0 ≤ 7.0 → merge → [1.0, 7.0] time=12.0: 12.0 > 7.0 → push → [1.0, 7.0, 12.0] Result: 3 fleets ✓

■ Time Complexity	■ Space Complexity
O(n log n)	O(n)

Problem 38 Largest Rectangle in Histogram

■ Real-World Analogy & What Problem It Solves

Urban planners use this to find the largest rectangular billboard that fits in a city skyline. Computer vision uses it to find the largest rectangular region in a binary matrix. Database query optimizers use histogram analysis to estimate selectivity for query planning. The monotonic stack solution here is the key to solving 'Maximal Rectangle in Binary Matrix' too.

■ Problem Statement

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

■ Intuition & Pattern: Monotonic Increasing Stack

Key insight: for each bar, find the largest rectangle where THAT bar is the shortest (limiting height). The width extends from the first bar to the left that is shorter, to the first bar to the right that is shorter. A monotonic increasing stack gives us both left and right boundaries automatically: when we pop a bar (because current bar is shorter), the current index is its right boundary, and the new stack top is its left boundary.

■■ Golang Implementation

```
func largestRectangleArea(heights []int) int {

stack := []int{} // indices, monotonically increasing heights

maxArea := 0

heights = append(heights, 0) // sentinel to flush remaining stack


for i, h := range heights {

// Maintain increasing stack

for len(stack) > 0 && heights[stack[len(stack)-1]] > h {

height := heights[stack[len(stack)-1]]

stack = stack[:len(stack)-1]

width := i

if len(stack) > 0 {

width = i - stack[len(stack)-1] - 1

}

area := height * width

if area > maxArea {
```

```
maxArea = area

}

}

stack = append(stack, i)

}

return maxArea

}
```

■ Dry Run: heights=[2,1,5,6,2,3]

i=0,h=2: stack:[0] i=1,h=1: 1<2 → pop 0, height=2, left=none→width=1, area=2. 1≤1→push. stack:[1] i=2,h=5: push. stack:[1,2] i=3,h=6: push. stack:[1,2,3] i=4,h=2: 2<6→pop 3,height=6,width=4-2-1=1,area=6. 2<5→pop 2,height=5,width=4-1-1=2,area=10. 2≥2→push. stack:[1,4] i=5,h=3: push. stack:[1,4,5] i=6,h=0(sentinel): 0<3→pop 5,h=3,w=6-4-1=1,area=3. 0<2→pop 4,h=2,w=6-1-1=4,area=8. 0<1→pop 1,h=1,w=6(no left),area=6. maxArea = 10 ✓

■ Time Complexity	■ Space Complexity
O(n)	O(n)

■ Pattern

Monotonic Stack Pattern: Push indices. When invariant breaks, pop and compute — each element pushed/popped once = O(n). Used in: Daily Temperatures, Car Fleet, Largest Rectangle, Trapping Rain Water.

■ QUEUE

Problems 39–43 · BFS · Monotonic Deque · Multi-Source Spread

Problem 39 Implement Queue Using Stacks

■ Real-World Analogy & What Problem It Solves

This problem teaches a deep systems concept: you can simulate one abstract data type using another. In embedded systems with limited hardware, you might only have stack-based memory but need FIFO (First-In-First-Out) ordering for message queues, print spoolers, and task schedulers. Understanding this conversion clarifies WHY we have distinct data structures.

■ Real World

Real-World Use: Message queues (Kafka, RabbitMQ) use FIFO order. OS task schedulers use FIFO for same-priority tasks. Print spoolers process jobs in arrival order.

■ Problem Statement

Implement a FIFO queue using only two stacks. Supported operations: push(x), pop(), peek(), empty(). Amortized O(1) per operation.

■ Intuition & Pattern: Two-Stack Queue

Stack is LIFO; queue is FIFO. To reverse order, push all elements from stack1 into stack2 — this reverses the order, making the oldest element on top of stack2. Lazy transfer: only move elements from stack1→stack2 when stack2 is empty AND we need pop/peek. This amortizes cost: each element moves at most once from s1→s2 → O(1) amortized.

■■ Golang Implementation

```
type MyQueue struct {

inbox []int // push here

outbox []int // pop/peek from here

}


func Constructor() MyQueue { return MyQueue{} }


func (q *MyQueue) Push(x int) {

q.inbox = append(q.inbox, x)

}
```

```
func (q *MyQueue) transfer() {
if len(q.outbox) == 0 {
for len(q.inbox) > 0 {
top := q.inbox[len(q.inbox)-1]
q.inbox = q.inbox[:len(q.inbox)-1]
q.outbox = append(q.outbox, top)
}
}
}

func (q *MyQueue) Pop() int {
q.transfer()
val := q.outbox[len(q.outbox)-1]
q.outbox = q.outbox[:len(q.outbox)-1]
return val
}

func (q *MyQueue) Peek() int {
q.transfer()
return q.outbox[len(q.outbox)-1]
}

func (q *MyQueue) Empty() bool {
return len(q.inbox) == 0 && len(q.outbox) == 0
}
```

■ Dry Run: push(1),push(2),peek,pop,empty

push(1): inbox=[1], outbox=[] push(2): inbox=[1,2], outbox=[] peek: outbox empty → transfer → inbox=[], outbox=[2,1] outbox.top=1 → return 1 ✓ pop: outbox=[2,1] → pop → return 1, outbox=[2] empty: inbox=[] AND outbox=[2] → false ✓

■ **Time Complexity** ■ **Space Complexity**

```
O(1) amortized                    O(n)
```

Problem 40 Sliding Window Maximum

■ Real-World Analogy & What Problem It Solves

Streaming analytics systems (Apache Flink, Kafka Streams) constantly compute 'maximum over last k events' for metrics like peak CPU usage, max response time, or highest bid in a time window. Without the deque solution, you'd recompute max over k elements for every new window → O(nk). The monotonic deque brings this to O(n), which is the difference between real-time and delayed analytics.

> **■ Real World**
> Real-World Use: Network traffic peak detection, financial rolling max price, sensor data anomaly detection, game leaderboard rolling windows.

■ Problem Statement

Given an integer array nums and a sliding window of size k, return the max of each window as it slides from left to right.

■ Intuition & Pattern: Monotonic Deque

Maintain a deque (double-ended queue) of indices in DECREASING order of their values. For each new element: 1. Remove indices from the BACK that are smaller than current (they can never be max while current exists) 2. Remove from FRONT if the index is outside the window (too old) 3. Front of deque always holds the index of the current window's maximum Each element is added/removed from the deque at most once → O(n) total.

■■ Golang Implementation

```
func maxSlidingWindow(nums []int, k int) []int {

deque := []int{} // stores indices, front = max index

result := []int{}


for i, num := range nums {

// Remove elements outside the window from front

for len(deque) > 0 && deque[0] &lt; i-k+1 {

deque = deque[1:]

}

// Remove smaller elements from back (they're useless)
```

```
for len(deque) > 0 && nums[deque[len(deque)-1]] &lt; num {

deque = deque[:len(deque)-1]

}

deque = append(deque, i)

// Record max once first window is complete

if i >= k-1 {

result = append(result, nums[deque[0]])

}

}

return result

}
```

■ Dry Run: nums=[1,3,-1,-3,5,3,6,7], k=3

i=0,n=1: deque=[0] i=1,n=3: 3>1 → pop 0, push 1 → deque=[1] i=2,n=-1: push 2 → deque=[1,2]. i≥2 → result=[nums[1]]=[3] i=3,n=-3: push 3 → deque=[1,2,3]. result=[3,nums[1]]=[3,3] i=4,n=5: 5>-3 pop 3; 5>-1 pop 2; 5>3 pop 1 → deque=[4]. result=[3,3,5] i=5,n=3: push 5 → deque=[4,5]. result=[3,3,5,5] i=6,n=6: 6>3 pop 5; 6>5 pop 4 → deque=[6]. result=[3,3,5,5,6] i=7,n=7: 7>6 pop 6 → deque=[7]. result=[3,3,5,5,6,7] Final: [3,3,5,3,6,7] ✓

■ Time Complexity	■ Space Complexity
O(n)	O(k)

Problem 41 Binary Tree Level Order Traversal

■ Real-World Analogy & What Problem It Solves

BFS (Breadth-First Search) is how social networks find 'people you may know' (friends of friends), how Google Maps finds shortest routes (unweighted), how web crawlers discover pages level by level, and how network packets are broadcast. Level-order traversal IS BFS on trees — foundational to understanding all graph traversal.

■ Real World
Real-World Use: Social network degrees of separation, shortest-path GPS routing, web crawlers, org chart rendering, file system directory trees, HTML DOM traversal.

■ Problem Statement

Given the root of a binary tree, return the level order traversal of its nodes' values (i.e., left to right, level by level).

■ Intuition & Pattern: BFS with Queue

Use a queue (FIFO). Start with root. Each iteration: record current queue length (= nodes at this level), process exactly that many nodes, enqueue their children. This cleanly separates levels without needing level markers.

■■ Golang Implementation

```
func levelOrder(root *TreeNode) [][]int {
if root == nil {
return nil
}
result := [][]int{}
queue := []*TreeNode{root}

for len(queue) > 0 {
levelSize := len(queue)
level := make([]int, 0, levelSize)

for i := 0; i &lt; levelSize; i++ {
node := queue[0]
queue = queue[1:]
level = append(level, node.Val)
if node.Left != nil {
queue = append(queue, node.Left)
}
if node.Right != nil {
queue = append(queue, node.Right)
}
}
result = append(result, level)
}
return result
```

```
}
```

■ Dry Run: Tree [3,9,20,null,null,15,7]

queue=[3], process 1 node: pop 3 → level=[3], enqueue 9,20 → queue=[9,20] queue=[9,20], process 2 nodes: pop 9 → level=[9] (no children); pop 20 → level=[9,20], enqueue 15,7 queue=[15,7], process 2 nodes: pop 15→level=[15]; pop 7→level=[15,7] Result: [[3],[9,20],[15,7]] ✓

■ Time Complexity	■ Space Complexity
O(n)	O(n) queue

Problem 42 Binary Tree Right Side View

■ Real-World Analogy & What Problem It Solves

Imagine standing to the right of a tree and looking left — you see exactly one node per level. This models 'visible elements in a layered UI' (only the top-most layer at each depth is visible), rendering engines for tree-structured data, and game development for occlusion culling (which objects are visible from the camera's perspective).

■ Problem Statement

Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

■ Intuition: BFS, Last Node Per Level

Level-order BFS. At each level, the LAST node processed is the rightmost visible node. Simply record queue[-1].Val (last dequeued at each level) into the result.

■■ Golang Implementation

```
func rightSideView(root *TreeNode) []int {

if root == nil {

return nil

}

result := []int{}

queue := []*TreeNode{root}


for len(queue) > 0 {

levelSize := len(queue)

for i := 0; i &lt; levelSize; i++ {
```

```
node := queue[0]

queue = queue[1:]

if i == levelSize-1 { // last node at this level

result = append(result, node.Val)

}

if node.Left != nil {

queue = append(queue, node.Left)

}

if node.Right != nil {

queue = append(queue, node.Right)

}

}

}

return result

}
```

■ Dry Run: [1,2,3,5,null,null,4]

Level 0: process [1] → last=1 → result=[1] Level 1: process [2,3] → last=3 → result=[1,3] Level 2: process [5,4] → last=4 → result=[1,3,4] Result: [1,3,4] ✓

■ Time Complexity	■ Space Complexity
O(n)	O(n)

Problem 43 Rotting Oranges

■ Real-World Analogy & What Problem It Solves

This is the classic 'multi-source BFS' or 'infection spread' problem. It models: virus spreading through a contact network (each infected person spreads to neighbors), fire spreading through a forest grid (each burning cell ignites adjacent ones), network packet flooding, and social influence propagation. Multi-source BFS processes ALL sources simultaneously, giving the minimum spread time.

■ **Real World**
Real-World Use: Epidemiology simulation, wildfire spread modeling, network broadcast storm analysis, contamination spread in supply chains, social media viral spread modeling.

■ Problem Statement

In a grid: 0=empty, 1=fresh orange, 2=rotten orange. Every minute, rotten oranges rot all 4-directionally adjacent fresh oranges. Return the minimum time to rot all oranges, or -1 if impossible.

■ Intuition: Multi-Source BFS

Start BFS from ALL rotten oranges simultaneously (all are sources). Each BFS level = 1 minute of spreading. Count fresh oranges initially. Each time we rot a fresh orange, decrement count. When BFS ends: if freshCount=0 → return minutes elapsed, else -1 (isolated fresh oranges).

■■ Golang Implementation

```
func orangesRotting(grid [][]int) int {

rows, cols := len(grid), len(grid[0])

queue := [][2]int{}

fresh := 0


// Initialize: find all rotten oranges and count fresh

for r := 0; r &lt; rows; r++ {

for c := 0; c &lt; cols; c++ {

if grid[r][c] == 2 {

queue = append(queue, [2]int{r, c})

} else if grid[r][c] == 1 {

fresh++

}

}

}


dirs := [][2]int{{0,1},{0,-1},{1,0},{-1,0}}

minutes := 0


for len(queue) > 0 && fresh > 0 {

minutes++
```

```
size := len(queue)

for i := 0; i &lt; size; i++ {

cell := queue[0]

queue = queue[1:]

for _, d := range dirs {

nr, nc := cell[0]+d[0], cell[1]+d[1]

if nr >= 0 && nr &lt; rows && nc >= 0 && nc &lt; cols && grid[nr][nc] == 1 {

grid[nr][nc] = 2

fresh--

queue = append(queue, [2]int{nr, nc})

}

}

}

}

if fresh > 0 {

return -1

}

return minutes

}
```

■ Dry Run

Grid: [[2,1,1],[1,1,0],[0,1,1]] Init: queue=[(0,0)], fresh=6 Min 1: rot (0,1),(1,0) → fresh=4, queue=[(0,1),(1,0)] Min 2: rot (0,2),(1,1) → fresh=2, queue=[(0,2),(1,1)] Min 3: rot (2,1) → fresh=1, queue=[(2,1)] Min 4: rot (2,2) → fresh=0, queue=[(2,2)] fresh=0 → return 4 ✓

■ Time Complexity	■ Space Complexity
O(m×n)	O(m×n)

■ Pattern

Multi-Source BFS Pattern: Initialize queue with ALL sources at once. Each level = 1 'step' of propagation. Guarantees minimum steps to reach all reachable cells. Also applies to: 01-Matrix (distance to nearest 0), Pacific Atlantic Water Flow, etc.

Pattern Summary

HashMap / Set	Contains Duplicate, Two Sum, Group Anagrams, Valid Anagram, Longest Consecutive, Valid Sudoku
Two Pointers	Valid Palindrome, 3Sum, Container With Most Water, Trapping Rain Water, Remove Nth Node
Sliding Window	Best Time to Buy Stock, Longest Substring, Longest Repeating Replacement, Min Window Substring
Prefix / Suffix	Product of Array Except Self, Encode/Decode Strings
Linked List Pointers	Reverse List, Merge Sorted, Floyd's Cycle, Reorder List, Copy with Random
Monotonic Stack	Daily Temperatures, Car Fleet, Largest Rectangle, Sliding Window Maximum
Backtracking	Generate Parentheses, Word Search, Palindromic Substrings
BFS / Queue	Level Order Traversal, Right Side View, Rotting Oranges, Sliding Window Max
Dynamic Programming	Word Break, Longest Palindromic Substring, Climbing Stairs
Stack Simulation	Valid Parentheses, Min Stack, Evaluate RPN, Implement Queue

Keep practicing. Patterns become instinct.